Contents

1. Document Control 4

2. Introduction 5

2.1 Context 5

2.2 Key Stakeholders 5

2.3 Document Organization 6

2.3.1 Audience 6

2.3.2 Document Structure 7

3. Solution Needs 8

3.1 Architectural Principles 8

3.2 Architectural Techniques Leveraged 10

3.3 Architectural Constraints 12

3.4 Business View 14

3.4.1 Personas and Main Scenarios 14

3.4.2 Deliverables & Solution Elements 16

3.4.3 Out of scope 17

3.5 Quality Attribute view 18

3.5.1 User Experience and Visual Design 18

3.5.2 Performance 19

3.5.3 Scalability 19

3.5.4 Availability and Reliability 19

3.5.5 Security 20

3.5.6 Maintainability and Evolvability 20

3.5.7 Operability and Monitoring 21

3.5.8 Auditability and Compliance 21

4. Solution Architecture 21

4.1 System Context View 21

4.2 Functional Blocks 24

4.2.1 Admin Control Portal 26

4.2.2 Web Portal 29

4.2.3 Partner / System APIs 31

4.2.4 OTP Verification Endpoint 33

4.2.5 Inquiry Management 34
4.2.6 Rules & Reference Data (Configuration) 38
4.2.7 QR Verification & Document Authenticity 40
4.2.8 Dashboards & Reporting 42
4.2.9 Integration Hub 44
4.2.10 Identity Hub (Access Management) 51
4.2.11 Notification Manager 53
4.2.12 Audit Trail 56
4.2.13 Logging & Tracing 59
4.2.14 Health Check & Service Status 62
4.3 Build View 64
4.3.1 Implementation Pattern 64
4.3.2 Technology stack 68
4.3.3 Team Responsibilities 70
5. Deployment View 73
5.1 Environments 73
5.2 Host configuration 74
Operating System Standard 75
Post-Installation Host Configuration Requirements 75
Hosting Configuration Objectives 76
6. Operation View 77
6.1 DevOps Enablement and Operational Tooling 77
6.2 Inner Development Loop and Engineering Practices 78
6.3 Feedback Loop Between Operations and Development 79
7. Hardware Sizing 81
7.1 Assumptions 81
7.2 Production environment 83
7.3 Pre-Production Environment (Optional / Preferred) 83
7.4 Staging / System Integration Testing (SIT) Environment 84
8. Appendices 85
8.1 Glossary of Key Terms 85
8.2 Use of Open Source 87
8.3 Licenses and subscriptions 87

8.4 Technologies & Techniques Used 88
8.4.1 Angular 88
8.4.2 .NET Core (ASP.NET Core) 88
8.4.3 Microsoft SQL Server 89

1. Document Control

Change Record:

Date	Author	Version	Change Ref.
05-JAN-2026	Mohamed Talaat	0.1	Initial draft
20-JAN-2026	Mohamed Talaat	1.0	Final version

Reviewers:

Reviewer Name	Version Approved	Position	Date

2. Introduction

This High-Level Design (HLD) document presents the proposed technical architecture for the **Security Clearance Gateway** solution. It defines the architectural structure of the system, including its major components, integration boundaries, deployment model, and operational considerations.

The document translates the approved business requirements into a coherent architectural view that supports scalability, security, auditability, and future extensibility. It serves as a technical reference for stakeholders involved in design review, implementation, deployment, and operation of the solution.

2.1 Context

The **Security Clearance Gateway** is a centralized digital platform designed to automate and govern the security inquiry and clearance process for travelers arriving in Egypt from designated countries or nationalities.

Currently, security inquiry procedures depend on fragmented, manual, and loosely coordinated channels between multiple governmental entities. This results in delays, inconsistent decision application, limited visibility, and high operational overhead. The proposed solution introduces a unified system that enables individuals and travel agencies to submit security inquiries digitally, while allowing governmental authorities to process, validate, and decide on these inquiries through a standardized and automated mechanism.

This High-Level Design (HLD) document describes the target technical architecture of the Security Clearance Gateway, translating approved business requirements into architectural building blocks, integration patterns, deployment topology, and operational considerations.

Client: Gahez

2.2 Key Stakeholders

Document is created as one of the deliverables of the solution implementation project and is to be reviewed and approved by the representatives of the technical committee.

Specifically, the main set of stakeholders from customer side are the members of the technical committee of the project. Solution Needs section, especially the business view subsection, is the one summarizing the solution requirements and business process approaches that might be of interest to business committee as well.

Client-Side Stakeholders (Gahez):

- Business Owner / Project Sponsor

- Security Operations Leadership
- IT & Infrastructure Teams
- Information Security & Compliance
- Finance and Audit Teams
- Decision Makers and Senior Management

Delivery-Side Stakeholders:

- Solution Architect (HLD Owner)
- Technical Leads
- DevOps / Release Engineers
- Quality Control & QA Team
- Operations and Support Teams

These stakeholders collectively contribute to defining requirements, approving architectural decisions, implementing the solution, and operating the system in production.

2.3 Document Organization

2.3.1 Audience

This section mentions Primary intended audience for the document.

The **Security Clearance Gateway** solution is intended to serve the following audience categories:

- **Travelers from selected nationalities**
  Individuals who do not hold a visa exemption for Egypt and are required to obtain a security clearance prior to travel. The applicable nationalities and countries are dynamically configured by authorized administrators based on governmental directives.
- **Travel Agencies**
  Authorized agencies responsible for organizing travel to Egypt for individuals or groups belonging to specific nationalities. Agencies may submit individual or batch inquiries on behalf of travelers and manage prepaid wallet balances for processing fees.
- **Governmental and Semi-Governmental Service Providers**
  External entities that participate in the security clearance process by providing inquiry responses or validations through system-to-system integrations.
- **Authorized Administrative Users**
  Government personnel responsible for managing configurations, monitoring inquiries, overseeing operations, and accessing dashboards and audit logs.

This audience definition reflects the operational scope of the solution and the roles interacting with it directly or indirectly.

2.3.2 Document Structure

This document is structured to present the architecture of the solution in a logical and progressive manner:

- **Section 1 – Document Control:** Versioning, authorship, and approval tracking.
- **Section 2 – Introduction:** Context, stakeholders, and document organization.
- **Section 3 – Solution Needs:** Architectural principles, constraints, business drivers, and quality attributes.
- **Section 4 – Solution Architecture:** System context, functional blocks, and build view.
- **Section 5 – Deployment View:** Target environments and host configuration.
- **Section 6 – Operation View:** Operational aspects such as monitoring, logging, and support.
- **Section 7 – Hardware Sizing:** Assumptions and initial production sizing.
- **Section 8 – Appendices:** Glossary, open-source usage, licenses, and technologies used.

This structure ensures clear traceability from business needs to architectural decisions and operational considerations.

3. Solution Needs

This section details the Solution Needs and considers the following items:

- Architectural principles.
- Architectural techniques leveraged.
- Architectural constraints.
- Business view.
- Quality attribute view.

3.1 Architectural Principles

Architectural Principles are long-lasting and rarely changing statements and guidelines that govern the design, implementation and operation of the solution.

Below Table details the Architectural Principles that govern the solution:

Principles	Rationale / Source	Implications
Security by Design	The solution processes sensitive traveller information and security clearance decisions that are subject to strict regulatory and governmental controls.	All components must enforce strong authentication and authorization, encrypt data in transit and at rest, secure all system-to-system communications, and maintain comprehensive audit logs for security-sensitive operations.
Automation First (Phase 1)	The solution is designed to operate without human intervention in the clearance decision lifecycle, relying on automated red-flag–based decision logic.	The architecture must support fully automated end-to-end processing, deterministic status transitions, background processing, retry mechanisms, and timeout handling without introducing manual workflow dependencies.
Configuration-Driven Policies and Rules	Clearance rules, nationalities, fees, inquiry eligibility, and integrations are subject to change based on governmental directives and operational needs.	Business policies must be externalized from code and managed through administrative configurations, supported by validation, versioning, and full auditability of configuration changes.
Modularity by Business Capability	The solution comprises distinct business capabilities such as inquiry submission, administration and control, clearance	The system must be structured into well-defined functional modules with clear responsibilities and interfaces, minimizing coupling and

	processing, document issuance, and reporting.	enabling controlled evolution of each capability.
Clean Architecture	Clean Architecture supports long-term maintainability, testability, and clear separation of concerns for complex and evolving systems.	The implementation must enforce layered separation between presentation, application logic, domain logic, and infrastructure concerns, with dependency flow directed inward toward the domain core.
Domain-Driven Design (Pragmatic Adoption)	The business domain includes well-defined concepts such as Inquiry, Batch, Clearance Decision, Red Flag, Nationality Rules, Agency Wallet, and Approval Document.	Core domain concepts must be explicitly modelled using domain entities and value objects with consistent ubiquitous language, while applying DDD selectively to avoid unnecessary complexity.
Selective Use of CQRS	The system combines transactional workloads with read-intensive dashboards and analytical views.	CQRS may be applied selectively to separate write operations from optimized read models for dashboards and reporting, without enforcing full CQRS or event sourcing across all modules.
Auditability and Traceability	Security oversight, compliance, and financial operations require full visibility into system behaviour and decisions.	All critical actions, including clearance decisions, configuration updates, wallet adjustments, integrations, and access events, must be logged with timestamps, actors, and correlation identifiers, and made available for audit and reporting.
Resilient Integration Design	The solution depends on integrations with external governmental and service-provider systems that may experience delays or temporary unavailability.	Integration components must implement timeout controls, retry strategies, error isolation, and graceful degradation to prevent external failures from impacting overall system stability.
Scalability for Batch Processing	Travel agencies may submit large batch inquiries and request bulk document downloads.	The architecture must support controlled parallel processing, background job execution, throttling, and batch-level tracking to handle peak workloads efficiently.

Extensibility and Integration Hub Orientation	The solution is expected to integrate with additional governmental and non-governmental entities over time.	The architecture must support extensibility through an integration hub or adapter-based approach, enabling new external systems to be onboarded with minimal impact on core business logic and without disrupting existing integrations.
Architecture Style Selection	Phase 1 scope, operational constraints, and integration complexity favor simplicity and controlled governance.	A modular monolithic architecture is recommended for Phase 1, with strong internal module boundaries and explicit contracts, while remaining ready for future evolution toward SOA or microservices if required.

3.2 Architectural Techniques Leveraged

This section outlines the key architectural techniques leveraged in the **Security Clearance Gateway** to support performance, scalability, reliability, and maintainability. The selected techniques are applied selectively based on solution needs and are not enforced uniformly across all components.

Cache-Aside Pattern

Data that is frequently accessed and rarely changed, such as nationality rules, fee configurations, inquiry status lookups, and reference data, is loaded on demand into an in-memory cache.

The application first checks the cache before accessing the primary data store. If the data is not found, it is retrieved from the database and stored in the cache for subsequent requests.

This technique reduces database load, improves response times, and supports high read volumes from portals and dashboards.

Index Tables and Optimized Indexing

Indexes are created on database fields that are frequently referenced in search, filtering, and reporting operations, such as inquiry status, nationality, agency ID, batch ID, submission date, and decision outcome.

This technique improves query performance for operational screens and dashboards without introducing additional data duplication.

Materialized Views (Read-Optimized Data Models)

For read-heavy use cases such as dashboards, monitoring screens, and analytical reports, materialized views or pre-aggregated read models are used.

These views are periodically refreshed or updated asynchronously to reflect the latest system state, enabling fast data retrieval without impacting transactional workloads.

Selective CQRS (Command Query Responsibility Segregation)

CQRS is applied selectively where clear separation exists between transactional operations (commands) and read-intensive operations (queries).

Write operations such as inquiry submission, payment confirmation, clearance decision, and configuration updates are handled through transactional command paths, while dashboards and reports consume optimized query models.

Full CQRS with event sourcing is not enforced globally to avoid unnecessary complexity.

Asynchronous Processing and Background Jobs

Long-running or resource-intensive operations such as batch inquiry processing, clearance evaluation, PDF and QR document generation, and bulk downloads are executed asynchronously using background jobs.

This technique ensures responsiveness of the user-facing portals and prevents blocking synchronous request threads.

Idempotent Operations

Critical operations such as inquiry submission, payment callbacks, clearance decision updates, and document generation are designed to be idempotent.

This ensures system consistency when retries occur due to network failures, timeouts, or integration issues.

Versioning of APIs and Configuration

External and internal APIs are versioned to support backward compatibility as the system evolves.

Configuration entities such as clearance rules, nationality settings, and fee structures are version-controlled logically to ensure traceability and safe rollout of changes.

Integration Adapters and Anti-Corruption Layer

External governmental and service-provider systems are accessed through dedicated integration adapters.

This approach isolates core business logic from external system models and protocols, protects the domain from external changes, and simplifies onboarding of new integrations in the future.

Controlled Use of Event-Driven Communication

Event-based notifications are used internally for decoupled communication between modules, such as triggering document generation after approval or notifying downstream components of state changes.

Events are used for coordination and extensibility, not as a full event-sourcing mechanism.

Error Handling, Retries, and Timeout Policies

Standardized timeout thresholds, retry strategies, and error-handling mechanisms are applied to all external integrations and background processes.

This ensures resilience and prevents individual failures from propagating across the system.

3.3 Architectural Constraints

The Architectural Constraints are derived from specific stakeholder dictates and govern its construction by defining items with which it must comply or otherwise consider such as Authentication, Quality of data, secure infrastructure, etc.

Below Table details the Architectural Constraints that govern the solution:

Architectural Constraints	Rationale / Source	Implication

Automated Decision-Only Processing	The clearance decision lifecycle is designed to be fully automated using deterministic rules without human intervention.	The architecture must not include human workflow engines, manual approval steps, or override mechanisms within the clearance processing path. All state transitions must be system-driven.
External Integration Dependency	The solution relies on multiple external governmental and non-governmental entities for inquiry validation and clearance responses.	The architecture must be resilient to partial failures, latency, and unavailability of external systems, using timeouts, retries, circuit breakers, and isolation layers to protect core processing.
High Sensitivity of Personal and Security Data	The system processes highly sensitive personal, travel, and security-related data.	Data must be protected through strong access controls, encryption at rest and in transit, secure key management, strict segregation of duties, and hardened deployment environments.
Mandatory Auditability and Non-Repudiation	All security decisions, configuration changes, and financial-related actions must be traceable for compliance and investigation purposes.	The architecture must enforce immutable audit trails, correlation IDs across transactions, tamper-resistant logs, and controlled access to audit data.
Deterministic and Explainable Decision Logic	Clearance decisions must be consistent, reproducible, and explainable based on predefined rules.	Decision logic must be rule-based, versioned, and traceable to configuration state at the time of execution, avoiding probabilistic or opaque decision mechanisms.
Support for High-Volume Batch Processing	Travel agencies can submit large batches containing multiple inquiries in a single operation.	The architecture must support controlled parallelism, background job execution, throttling per agency, and batch-level tracking without impacting individual inquiry processing.

Configurability Without Runtime Downtime	Administrative changes to rules, fees, nationalities, and integrations must be applied dynamically.	The architecture must support hot configuration updates, validation mechanisms, and safe propagation of changes without requiring system downtime or redeployment.
Strong Isolation Between Core Domain and Integrations	External systems are owned by different authorities and evolve independently.	The core domain must be protected through integration adapters or an integration hub pattern, preventing external data models or failures from leaking into core business logic.
Consistent State Management Across Long-Running Processes	Inquiry processing may span multiple steps and external calls over extended time periods.	The architecture must maintain consistent state transitions, idempotent operations, and recovery mechanisms to avoid duplicated decisions, lost updates, or stuck inquiries.
Read-Heavy Operational and Analytical Workloads	Dashboards and reporting introduce significant read pressure alongside transactional workloads.	The architecture must separate read optimization concerns (indexes, read models, materialized views) from transactional processing to ensure system stability and performance.

3.4 Business View

3.4.1 Personas and Main Scenarios

The Security Clearance Gateway serves multiple personas that interact with the system through distinct functional paths. These personas and their associated business scenarios define the primary business drivers influencing the architecture.

Key Personas:

- **Traveler**: Submits individual security inquiries, completes required payments, tracks inquiry status, and downloads clearance documents when approved.
- **Travel Agency**: Submits individual or batch inquiries on behalf of travelers, manages prepaid wallet balances, and downloads approval documents in bulk.

- **Administrative User**: Manages system configurations such as nationalities, fees, inquiry rules, agencies, and integrated entities, and monitors operational activities.
- **Decision Maker**: Consumes dashboards and analytical views to gain visibility into inquiry volumes, trends, performance, and compliance indicators.
- **External Service Provider**: Participates in the clearance process by providing validation or response data through system-to-system integrations.

Main Scenarios (High-Level):

- Submission of individual security inquiries by travelers.
- Submission and processing of batch inquiries by travel agencies.
- Automated clearance processing and rule-based decision making.
- Generation and verification of approval documents (PDF with QR code).
- Configuration and governance of clearance rules and reference data.
- Operational monitoring, reporting, and audit review.

Detailed business flows, use cases, and scenario descriptions are documented in the **Business Requirements Document (BRD)** and are referenced here to provide architectural context without duplication.

3.4.2 Deliverables & Solution Elements

Solution deliverables will be as below:

- **Security Clearance Web Portal**
  - Individual inquiry submission
  - Agency inquiry submission (individual and batch)
  - Payment initiation for individuals
  - Wallet usage for agencies
  - Inquiry status tracking
  - Approval document download (PDF)
  - User account and profile management
- **Administration & Control Module**
  - Nationality and country configuration
  - Inquiry eligibility rules management
  - Fee configuration per nationality or country
  - Agency onboarding, approval, and management
  - Manual wallet balance adjustments (authorized users only)
  - Integrated entity management
  - System configuration and parameter management
- **Clearance Engine**
  - Automated inquiry intake
  - Rule-based clearance evaluation (red-flag logic)
  - Inquiry lifecycle and status management
  - Timeout, retry, and failure handling
  - Batch inquiry processing control
  - Decision finalization and outcome persistence
- **Integration Hub / Integration Layer**
  - Outbound integration with governmental entities
  - Inbound response handling and validation
  - Integration adapters per external entity
  - Timeout, retry, and error isolation mechanisms
  - Extensible onboarding of future governmental and non-governmental systems
- **Document Generation Service**
  - Approval document (PDF) generation
  - QR code generation and embedding
  - Document template configuration
  - Secure document storage and retrieval
  - Idempotent document creation and retry handling
- **QR Verification Endpoint**
  - QR code validation endpoint

    - Real-time approval status lookup
    - Controlled access to verification data
    - Tamper detection and document authenticity assurance
- **Notification Mechanism**
    - Inquiry submission notifications
    - Approval and rejection notifications
    - Payment and wallet transaction notifications
    - System-generated alerts for processing outcomes
    - Asynchronous notification dispatching
- **Dashboards & Reporting Components**
    - System-wide inquiry monitoring dashboard
    - Nationality and trend analytics
    - Agency performance and compliance dashboards
    - Financial and wallet analytics
    - Batch processing monitoring
    - Security and audit insight dashboards
- **Audit Logging Component**
    - Inquiry lifecycle audit trails
    - Clearance decision logging
    - Configuration change logging
    - Wallet and financial operation logs
    - Administrative access and activity logs
- **Configuration & Reference Data Store**
    - Nationality and country reference data
    - Inquiry rule definitions
    - Fee and pricing configurations
    - System parameters and thresholds
    - Versioned configuration records

3.4.3 Out of scope

Any area not explicitly included in the Deliverables & Solution Elements section is considered out of scope for this project.

In addition, the following areas are explicitly excluded from the scope of this phase:

Area	Comment
Mobile Applications	Native mobile applications for travelers or agencies are not included. The solution is limited to web-based access.
Risk Scoring or Predictive Analytics	Advanced risk scoring, behavioral analysis, or AI/ML-based decision-making mechanisms are not part of this phase.

Identity Verification Beyond Provided Data	Biometric verification, facial recognition, or document authenticity checks beyond provided inquiry data are not included.
External Payment Providers	Integration with payment providers other than the approved payment gateway is not supported in this phase.
Data Migration from Legacy Systems	Migration of historical data from existing or legacy systems is not part of the project scope.

3.5 Quality Attribute view

This section defines the key quality attributes that drive the architectural design of the Security Clearance Gateway. Each attribute is expressed in objective terms and linked directly to architectural responsibilities and constraints, rather than implementation details.

3.5.1 User Experience and Visual Design

The system must support multiple user categories with different responsibilities while ensuring clarity, predictability, and minimal operational friction.

- When users interact with the system, the interface must expose only the functions and data relevant to their assigned role, preventing unauthorized or unnecessary access paths.
- When a user submits an inquiry or performs an administrative action, the system must provide immediate feedback confirming receipt and current processing state.
- When inquiry processing occurs asynchronously, the system must communicate progress and outcomes clearly without requiring users to understand internal processing steps.
- The system must support **bilingual user interfaces (Arabic and English)** across all user-facing modules, with language selection applied consistently and without affecting business logic or data integrity.
- User interface behavior, terminology, and visual patterns must remain consistent across all functional modules to reduce learning effort and operational errors.
- User-facing components must be architecturally separated from core business logic to allow independent evolution of visual design, localization, and accessibility features without impacting clearance rules or integrations.

3.5.2 Performance

The system must support both **real-time interactions and high-volume** background processing without degrading responsiveness or stability.

- When travelers or agencies submit inquiries or view inquiry status, the system must remain responsive and must not be blocked by clearance processing, batch handling, or document generation.
- When agencies submit batch inquiries containing large numbers of travelers, the system must register the batch immediately and process individual inquiries asynchronously.
- When multiple inquiries are processed concurrently, the system must sustain predictable throughput without creating processing bottlenecks.
- When dashboards or monitoring screens are accessed during peak submission periods, read operations must not interfere with transactional processing.
- When external integrations respond slowly or experience latency, user-facing operations must remain unaffected.

These requirements are enforced through asynchronous processing, controlled parallelism, read-optimized access paths, and isolation of integration workloads.

3.5.3 Scalability

The architecture must support growth in usage, integrations, and data volume without requiring fundamental redesign.

- When inquiry volumes increase due to seasonal demand or agency activity, the system must scale processing capacity without impacting correctness or stability.
- When new governmental or non-governmental entities are integrated, the system must accommodate them without modifying core domain logic.
- When batch sizes increase, the system must scale background processing independently of user-facing components.
- When reporting and analytics usage grows, read workloads must scale independently from transactional processing.

Scalability is achieved through modular decomposition, background job execution, horizontal scaling capabilities, and separation of read and write concerns.

3.5.4 Availability and Reliability

The system must provide consistent and predictable behavior, even in the presence of failures or partial outages.

- When a component fails or restarts, the system must recover without data loss or inconsistent inquiry states.
- When external systems are unavailable, the system must continue operating and reflect accurate processing states rather than failing silently or blocking.
- When processing is interrupted, the system must resume safely without duplicating decisions or corrupting inquiry state.
- When failures occur, the system must surface them clearly for operational visibility.

Availability and reliability are ensured through persistent state management, retry and timeout mechanisms, failure isolation, and explicit processing state transitions.

3.5.5 Security

Security is a foundational quality attribute due to the sensitivity of personal and clearance-related data.

- When users access the system, all actions must be authenticated and authorized based on assigned roles.
- When administrative or decision-related actions occur, they must be traceable to a specific identity and execution context.
- When data is stored or transmitted, it must be protected against unauthorized access or tampering.
- When integrations are invoked, communication must be secured and limited to approved endpoints.
- When security-relevant events occur, they must be logged and available for audit and investigation.

Security controls are enforced consistently across application, integration, and infrastructure layers.

3.5.6 Maintainability and Evolvability

The system must support ongoing change driven by policy updates, integrations, and future phases.

- When clearance rules, fees, or eligibility criteria change, the system must adapt without requiring redeployment.
- When new capabilities are introduced, changes must be localized and not cascade across unrelated modules.
- When defects are corrected or enhancements are introduced, the impact scope must be predictable and limited.
- When the system evolves, architectural consistency must be preserved.

These requirements are supported through modular design, configuration-driven behavior, and clear separation of responsibilities.

3.5.7 Operability and Monitoring

The system must be observable and operable in production environments.

- When abnormal behavior occurs, operational teams must be able to detect, diagnose, and respond without system downtime.
- When processing delays or failures occur, they must be visible through logs and monitoring indicators.
- When operational thresholds are exceeded, alerts must be raised in a timely manner.
- When system health is reviewed, relevant metrics and logs must be available.

Operability is supported through centralized logging, monitoring hooks, and operational dashboards.

3.5.8 Auditability and Compliance

The system must support regulatory oversight and internal governance requirements.

- When clearance decisions are made, the rationale and execution context must be traceable.
- When configuration changes occur, historical versions and actors must be recorded.
- When financial-related actions are executed, they must be auditable and linked to system records.
- When audits are conducted, required evidence must be retrievable without system disruption.

Auditability is enforced through immutable logs, correlation identifiers, and controlled access to audit data.

4. Solution Architecture

This section describes our solution architecture (include diagrams) and how it help to achieve business objectives.

4.1 System Context View

The System Context View illustrates the **Security Clearance Gateway** as a single logical system and shows its interactions with external actors and external systems. This view defines the **solution boundary**, identifies **trust boundaries**, and clarifies **high-level responsibilities** without exposing internal implementation details.

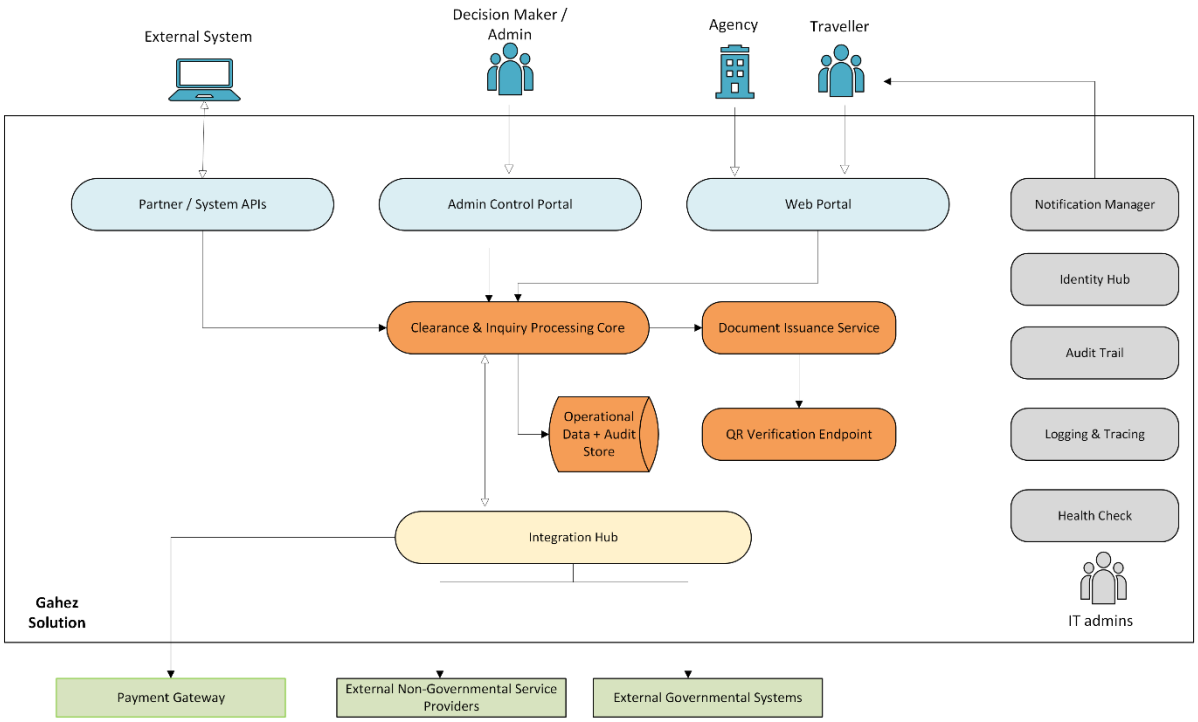

Figure 1 Context Diagram (Level 0)

Diagram Description

The diagram presents the Security Clearance Gateway at the center, enclosed within a clear system boundary. All users and systems outside this boundary are considered external and interact with the solution through controlled and secured interfaces.

External Actors (as shown in the diagram)

- **Travelers**
  Submit individual security clearance inquiries, perform payments when applicable, track inquiry status, and download approval documents.
- **Travel Agencies**
  Submit individual or batch inquiries on behalf of travelers, manage prepaid wallet balances, and retrieve approval documents in bulk.
- **Administrative Users**
  Configure clearance rules, manage reference data, control integrations, and monitor operational activities.
- **Decision Makers**
  Access dashboards and analytical views for monitoring trends, performance indicators, and compliance metrics.

External Systems (as shown in the diagram)

- **External Governmental Systems**
  Government-owned systems that participate in the clearance process by providing validation, approval, or rejection responses.
- **External Non-Governmental Service Providers**
  External entities that provide supplementary information or services required during clearance processing.
- **Payment Gateway**
  External service responsible for processing individual payments and agency wallet top-ups.

High-Level Interaction Flows

The context diagram highlights the following interaction flows:

- Inquiry submission and status tracking between users (travelers and agencies) and the solution.
- Configuration and monitoring interactions between administrative users and the solution.
- Secure system-to-system exchanges between the solution and external governmental and non-governmental systems.
- Payment transaction initiation and confirmation between the solution and the payment gateway.
- Reporting and analytics consumption by decision makers.

Trust Boundary and Responsibility Allocation

- The **solution boundary** encapsulates all inquiry processing, automated decision logic, rule enforcement, document generation, and audit logging.
- All **external systems** are treated as untrusted and accessed through secure, validated integration interfaces.

- The solution is responsible for:

  - Managing inquiry lifecycle and state
  - Executing automated clearance decisions
  - Enforcing configuration and security policies
  - Maintaining auditability and traceability

- External systems are responsible for:

  - Providing accurate and timely responses
  - Maintaining availability of their exposed interfaces
  - Enforcing their own internal governance and security controls

4.2 Functional Blocks

Through this section we will dive deeper into core blocks of solution modules. each block should be described into a separate section

This section decomposes the solution into major functional blocks and clarifies the responsibilities of each block. The purpose is to present a clear functional breakdown that supports the build view and deployment view, while remaining independent of implementation details.

The solution is implemented as a set of modules within a single solution boundary. These modules are organized into:

- Presentation blocks,
- Core domain/business blocks,
- Integration block,
- Platform/support blocks,
- Observability and operations blocks.

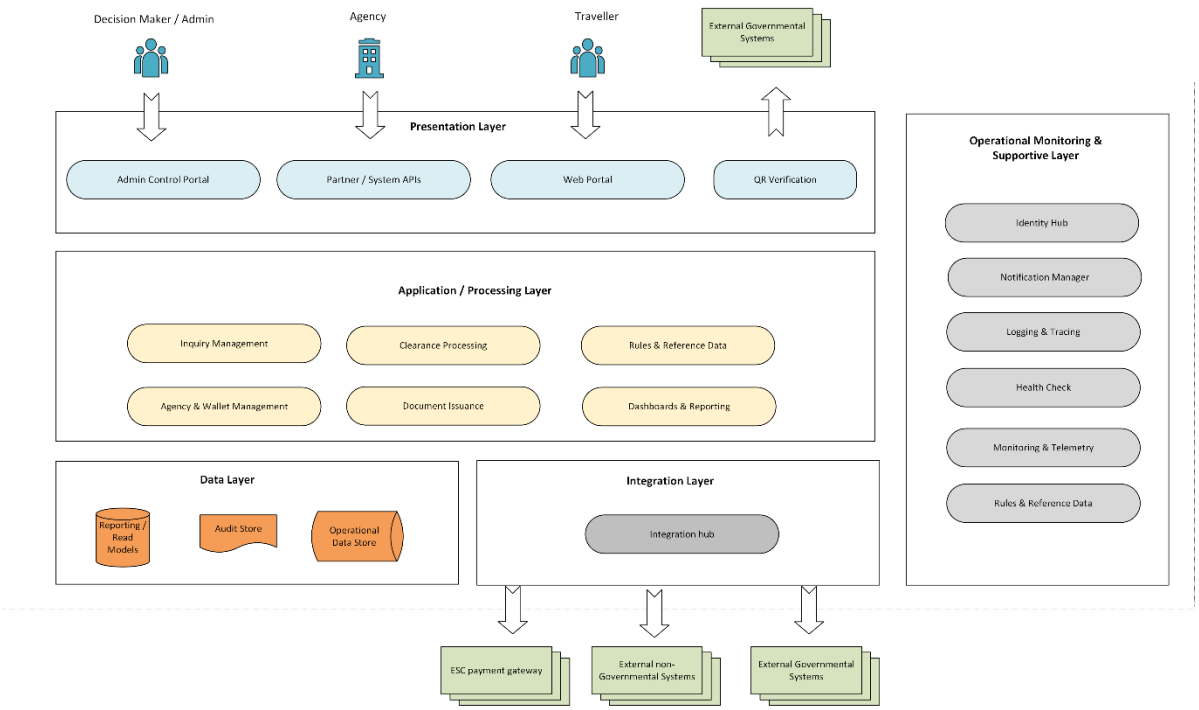

Figure 2 Functional Block diagram

- **Presentation Layer**

  - **Web Portal**
    - Traveler and agency self-service functions (inquiry submission, status tracking, document retrieval)
  - **Admin Control Portal**

    - Administrative configuration, dashboards, reporting, and oversight functions
  - **Partner / System APIs**
    - Agency and partner integrations (submission, status APIs, batch operations, wallet operations)
  - **QR Verification Endpoint**
    - Verification endpoint for approval documents using QR code

- **Core Business and Domain Modules**

  - **Inquiry Management Module**
    - Inquiry capture, validation, lifecycle state management, and persistence
    - Batch submission registration and tracking
  - **Clearance Processing Module**
    - Automated decision processing using rule-based logic (red-flags)
    - Orchestrates processing steps and transitions inquiry state deterministically
  - **Rules & Configuration Module**
    - Manages configurable elements influencing processing (nationalities, fees, eligibility parameters, integration settings)
    - Provides versioning and controlled updates of configuration
  - **Agency & Wallet Module**
    - Agency onboarding and profile management
    - Wallet top-up, balance management, and wallet transaction recording
  - **Payment Module**
    - Payment initiation and confirmation processing
    - Reconciliation linkage between payment outcomes and inquiry lifecycle
  - **Document Issuance Module**
    - Approval document generation (PDF) and QR embedding
    - Document storage and controlled retrieval
  - **Reporting & Dashboards Module**
    - Operational dashboards and analytical views for decision makers and administrators
    - Read-optimized views and aggregation support

- **Integration Block**

  - **Integration Hub Module**
    - Standardized integration interfaces and adapters for external governmental and non-governmental systems
    - Request/response handling, validation, failure isolation, and extensibility for new integrations

- **Platform / Support Modules (Modules, not Microservices)**

- **Identity Hub (Access Management)**
  - Authentication integration, role and permission mapping, authorization enforcement points
- **Notification Manager**
  - Notification dispatch orchestration (templates, channels, retry policies, and delivery tracking)
- **Audit Trail**
  - Immutable audit records for sensitive actions and decisions (who/what/when/context)
- **Logging & Tracing**
  - Centralized structured logging, correlation identifiers, and traceability across modules
- **Health Check**
  - Liveness/readiness checks and dependency checks (database, integrations, background processing)

- **Operational Monitoring & Telemetry**

- **Monitoring & Telemetry Module**
  - Metrics, service health indicators, alerts, and operational observability hooks

4.2.1 Admin Control Portal

Purpose

The Admin Control Portal is the primary interface for administrative users and decision makers to configure the solution, control master data, monitor operations, and access dashboards and reports. It is designed to support governance, operational oversight, and controlled configuration changes.

Primary Users

- Administrative Users: manage configuration, integrations, operational follow-up, and compliance evidence.
- Decision Makers: consume dashboards, reports, and oversight analytics.

Key Responsibilities

- Provide secure, role-based access to administrative and oversight functions.
- Enable controlled configuration of business rules, reference data, and external entity parameters.

- Provide operational visibility into inquiry lifecycle states, processing backlogs, and integration status.
- Provide dashboards and reports for business and operational monitoring.
- Support auditability by recording administrative actions and decision-related events.

Sub-Systems and Functional Areas

- Configuration Management
  - Eligibility configuration (nationalities, conditions, parameters)
  - Fees and pricing configuration
  - Service configuration (SLA parameters, operational thresholds if applicable)
  - Templates configuration (document templates, notification templates where applicable)
- Rules & Reference Data Administration
  - Maintain reference lists and lookup data used by processing modules
  - Controlled updates and versioning of reference items when required
  - Activation/deactivation of specific rules/criteria based on policy
- Integration Administration
  - Manage external entity profiles (governmental/non-governmental)
  - Configure integration endpoints and parameters
  - View integration health indicators and error summaries (high-level)
- Operations Console
  - Search and view inquiries (by reference, passport, agency, date range, status)
  - View inquiry lifecycle and processing status
  - Review exception cases and failure reasons (without exposing sensitive internals)
  - Monitor batch processing and queue/backlog indicators (if batch processing exists)
- Dashboards & Reporting
  - Operational dashboards: volumes, turnaround, outcomes distribution, backlog trends
  - Financial dashboards: payments, wallet transactions, settlement indicators (high-level)
  - Compliance dashboards: audit activity summaries, administrative actions overview
- User & Access Administration (via Identity Hub)
  - Role assignment / permission mapping (as governed by Identity Hub)
  - Administrative access oversight (who has access to what)

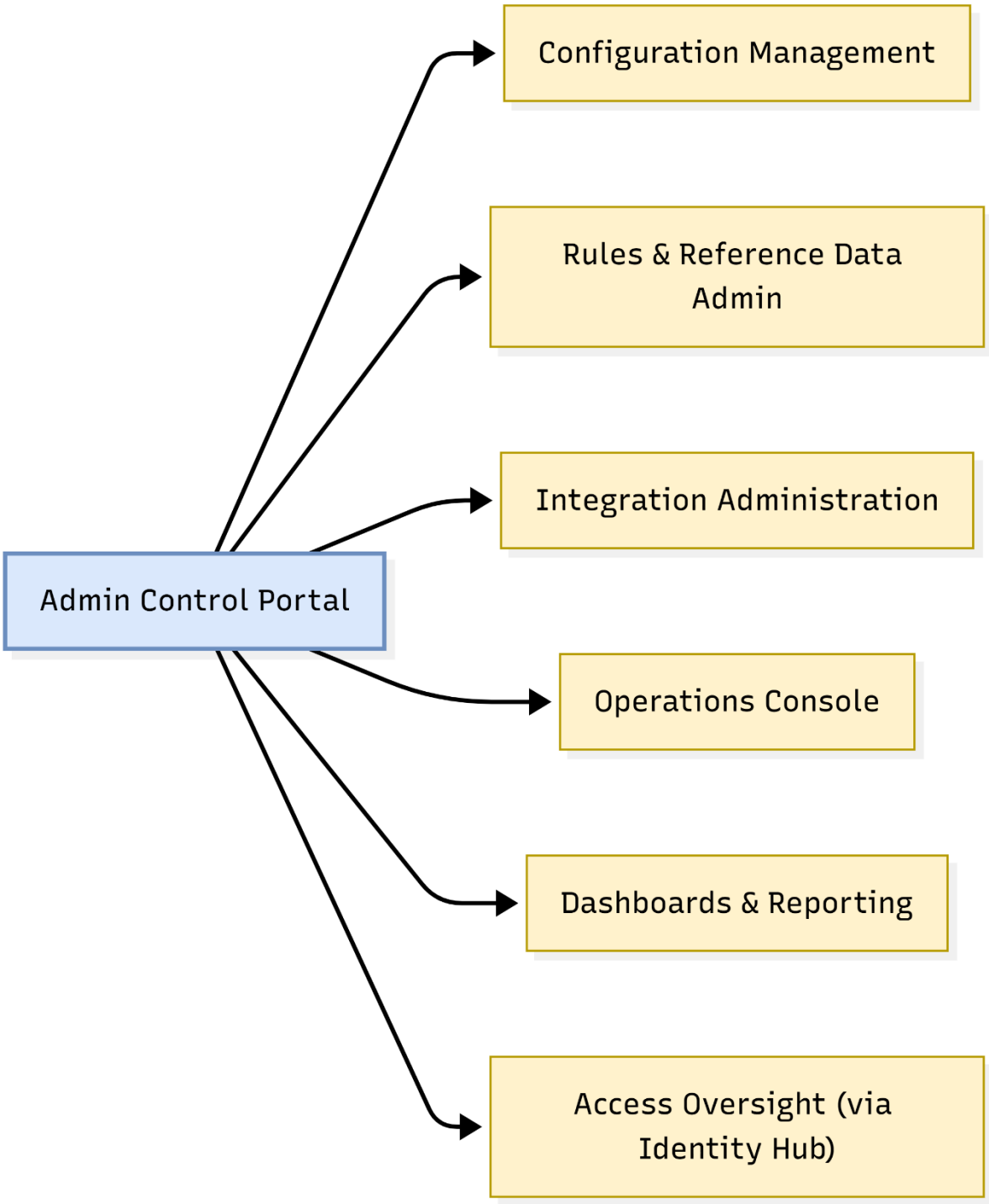

Figure 3 Admin Control sub-systems

While The following sequence diagram illustrates a representative administrative operation performed through the Admin Control Portal. The example demonstrates how a configuration change (such as updating eligibility rules or fees) is processed, validated, and audited.

This flow is provided for illustration purposes to clarify the interaction between the Admin Control Portal and the supporting platform modules. It does not represent all possible administrative operations, nor does it define implementation-level details.

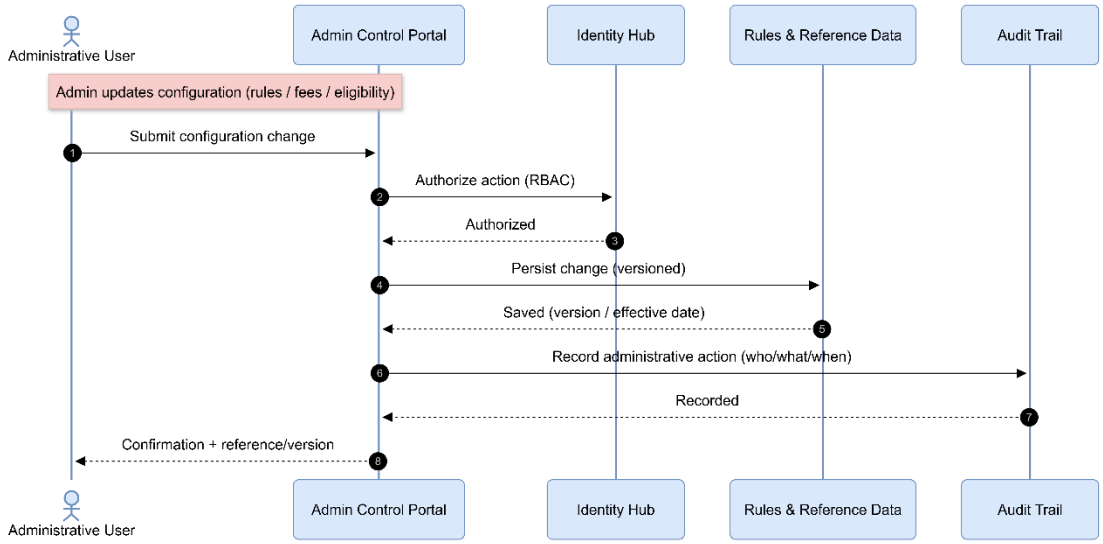

Figure 4 Configuration Change

4.2.2 Web Portal

Purpose

The **Web Portal** is the primary user-facing channel of the solution, providing self-service capabilities for **travelers** and **travel agencies**.
It enables users to submit security clearance inquiries, track their processing status, initiate payments through the external payment gateway, and retrieve final approval documents.

The Web Portal is designed to provide a secure, intuitive, and bilingual (Arabic / English) experience while enforcing eligibility rules and access controls defined by the solution.

Primary Users

- **Travelers**
  - Individuals submitting security clearance inquiries for personal travel purposes.
- **Travel Agencies**
  - Authorized agencies submitting inquiries on behalf of travelers (individual or batch).
  - Agencies managing prepaid wallets and bulk document retrieval.

Key Responsibilities

- Provide secure access for travelers and agencies through role-based authentication.
- Capture inquiry data accurately and validate it before submission.
- Support both **individual** and **batch** inquiry submission flows.
- Enable users to track inquiry status throughout the processing lifecycle.
- Initiate payment transactions through the **external ESC Payment Gateway**.
- Allow retrieval of approval documents once processing is completed.
- Ensure a consistent, bilingual user experience (Arabic / English).
- Enforce business rules and eligibility constraints at the user interface level (without embedding business logic).

Sub-Systems and Functional Areas

Traveler Self-Service

- Individual inquiry submission
- Inquiry status tracking
- Payment initiation (when applicable)
- Download of approval documents (PDF)
- QR-based verification redirection (if applicable)

Agency Self-Service

- Individual inquiry submission
- Batch inquiry upload and tracking
- Wallet balance view and transaction history
- Wallet top-up initiation (via external payment gateway)
- Bulk download of inquiry results and documents

User Account & Profile Management

- Profile view and basic account settings
- Language preference (Arabic / English)
- Session and access management (via Identity Hub)

Inquiry Tracking & Notifications

- Real-time or near real-time status updates
- Visibility of processing stages (submitted, under review, completed, rejected)
- Notification triggers (submission, payment confirmation, final decision)

4.2.3 Partner / System APIs

Purpose

The **Partner / System APIs** provide a secure, programmatic interface for **authorized external parties**, primarily **travel agencies** and **integrated service providers**, to interact with the solution without using the Web Portal user interface.

These APIs enable system-to-system integration for inquiry submission, batch processing, status retrieval, wallet operations, and document access, while enforcing the same business rules, security controls, and audit requirements applied to interactive channels.

Primary Users

- **Travel Agencies (System-to-System)**
  - Agencies integrating their internal systems directly with the solution.
- **Authorized External Service Providers**
  - Third-party systems approved to submit or query inquiries programmatically.

Key Responsibilities

- Expose secure APIs for inquiry submission (individual and batch).
- Provide APIs for inquiry status tracking and result retrieval.
- Support agency wallet operations (balance inquiry, top-up initiation).
- Enable bulk retrieval of inquiry outcomes and documents.
- Enforce authentication, authorization, and throttling policies.
- Ensure functional parity with Web Portal business capabilities.
- Provide predictable and versioned API contracts for long-term integration stability.

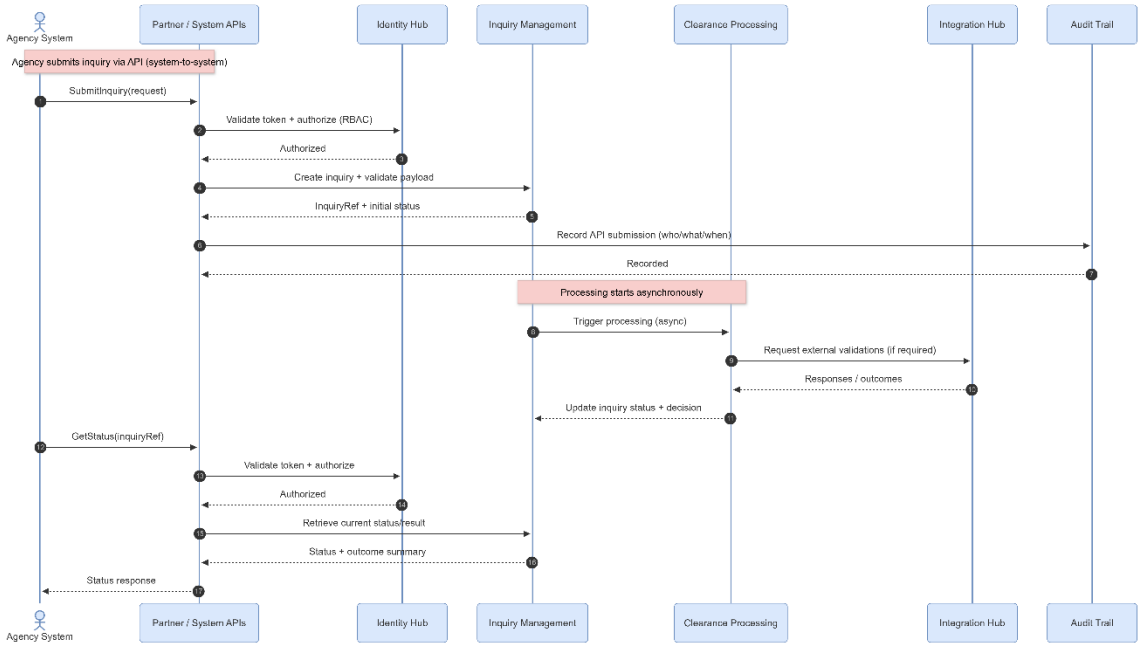

Figure 5 sequence diagram for Agency inquiry

Sub-Systems and Functional Areas

Inquiry APIs

- Individual inquiry submission
- Batch inquiry submission and tracking
- Inquiry status retrieval
- Inquiry result retrieval

Wallet APIs (Agency Only)

- Wallet balance inquiry
- Wallet transaction history
- Wallet top-up initiation (via external payment gateway)

Document APIs

- Download approval documents (PDF)
- Bulk document retrieval for batch submissions
- QR verification reference retrieval (if applicable)

API Security & Governance

- API authentication and authorization (via Identity Hub)
- Request validation and schema enforcement
- Rate limiting and throttling

- API versioning and backward compatibility management

4.2.4 QR Verification Endpoint

Purpose

The **QR Verification Endpoint** provides a lightweight and secure mechanism to **verify the authenticity and validity of issued clearance documents**.
It is primarily intended for **external verifiers** (e.g., airport staff, border control officers, or authorized personnel) to validate a document by scanning a QR code embedded within the issued approval document.

The endpoint is designed for **read-only verification** and does not expose sensitive personal or processing details.

Primary Users

- **External Verifiers**
  - Airport or border control personnel
  - Authorized inspection entities
- **Internal Support Users (Read-only)**
  - Operational staff performing verification checks (if applicable)

Key Responsibilities

- Accept verification requests initiated through QR code scanning.
- Validate the referenced document and its current status.
- Retrieve verification data from the document and inquiry records.
- Present a clear and unambiguous verification result (valid / invalid / expired / revoked).
- Prevent tampering, replay attacks, and unauthorized data exposure.
- Ensure high availability and fast response times due to real-time usage scenarios.

Sub-Systems and Functional Areas

QR Reference Resolution

- Decode and validate QR reference identifiers.
- Validate request structure and integrity.
- Enforce expiration and reuse rules (if applicable).

Document Validation

- Verify document existence and integrity.

- Confirm document issuance status and validity period.
- Detect revoked, expired, or superseded documents.

Verification Response Rendering

- Display verification result in a simplified format.
- Avoid exposure of sensitive personal or processing data.
- Support bilingual output (Arabic / English), if required.

Security Controls

- Input validation and tamper detection.
- Rate limiting to prevent abuse.
- Minimal data exposure by design.

4.2.5 Inquiry Management

Purpose

The **Inquiry Management** module is responsible for managing the full lifecycle of a **Security Clearance Inquiry** from initial submission until closure.
It acts as the system's authoritative source for inquiry state, ensuring that inquiries are validated, persisted, tracked, and made available for processing, status retrieval, and document issuance.

This module provides consistent inquiry handling regardless of the submission channel (Web Portal or Partner/System APIs).

Primary Users

- **Travelers** (through Web Portal)
- **Travel Agencies** (through Web Portal and/or Partner/System APIs)
- **Administrative Users** (through Admin Control Portal for monitoring and operations)
- **Downstream Modules**
  - Clearance Processing
  - Document Issuance
  - Reporting & Dashboards

Key Responsibilities

- Capture inquiry submissions (individual and batch) and validate required data.
- Generate inquiry identifiers and manage inquiry lifecycle state transitions.
- Maintain inquiry status and make it available for user-facing tracking.

- Enforce baseline eligibility checks and completeness checks (input-level validations).
- Trigger clearance processing workflows according to configured rules.
- Persist inquiry data and lifecycle events in the Data Layer.
- Provide search and retrieval functions for operational monitoring and reporting.
- Produce audit events for critical lifecycle actions (submit, update, decision, document issued).

Sub-Systems and Functional Areas

Inquiry Capture & Validation

- Validate required fields and formats (e.g., passport identifiers, nationality, travel details)
- Enforce basic data consistency rules (e.g., date ranges, required attachments if applicable)
- Protect against duplicate or replay submissions (where applicable)

Individual Inquiry Lifecycle Management

- Create and persist inquiry records
- Manage status transitions (e.g., Submitted → Under Processing → Completed / Rejected)
- Maintain timestamps, SLA indicators, and processing markers

Batch Inquiry Management (Agency)

- Register batch uploads and batch references
- Track per-item statuses inside a batch
- Provide batch-level progress indicators and bulk retrieval readiness

Inquiry Status & Tracking Services

- Provide status endpoints for Web Portal and APIs
- Provide consistent status mapping for UI/API consumption
- Support advanced inquiry lookup for administrators (filters, search, and trace references)

Processing Triggering & Orchestration Hooks

- Trigger Clearance Processing asynchronously upon successful submission
- Support reprocessing rules for exceptional or administrative scenarios (if allowed by policy)

- Coordinate processing sequencing with Integration Hub dependencies

Payment Awareness (External Integration Dependency)

- Mark inquiries as "payment required / pending / confirmed" based on callbacks from external payment gateway
- Ensure processing is gated appropriately when payment is mandatory (policy-driven)

Queueing and Asynchronous Processing

To ensure responsive user experience and predictable processing under peak load, the solution executes clearance processing as **asynchronous background jobs**. Upon successful submission, Inquiry Management persists the inquiry record and schedules processing using **Hangfire** (SQL Server–backed job storage).

High-level approach

- Inquiry submission returns an **inquiry reference** immediately after validation and persistence.
- Inquiry Management enqueues a processing job in Hangfire.
- Background workers execute processing jobs and update inquiry status deterministically.
- All processing outcomes, events, and status transitions are persisted for traceability and reporting.

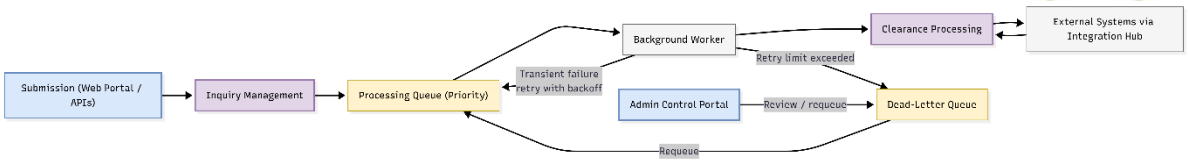

Figure 6 Queue & Retry

Priority Model (SLA-Driven)

Each inquiry is assigned a priority at submission time based on **traveler/agency selection** and aligned with the solution's **SLA targets**.

Priority levels

- **Express (P1)**: prioritized processing path for time-sensitive requests (policy-controlled availability).
- **Standard (P2)**: default processing level.
- **Batch (P3)**: applied to batch items; processed without starving interactive requests.

Priority enforcement

- Priority is stored within the inquiry record and reflected in the Hangfire queue name.
- Priorities are used exclusively for scheduling and SLA control; decision logic remains unchanged.

Hangfire Queue Design (Logical Queues)

Hangfire is configured with multiple queues to support priority and fairness:

- express (P1)
- standard (P2)
- batch (P3)

Inquiry Management schedules the processing job to the appropriate queue based on the selected priority.

Batch Fairness and Worker Allocation Cap

To prevent batch processing from consuming all capacity and impacting SLA for interactive requests, the solution applies a worker allocation policy:

- **Batch Processing Cap (recommended): max 30% of workers**
  - 70% of workers consume express + standard
  - 30% of workers consume batch

This approach ensures continuous batch progress while preserving capacity for SLA-sensitive interactive work. The cap is configurable and can be tuned using operational telemetry (queue backlog, average lead time, SLA adherence).

Retry Mechanisms (Safe and Bounded)

A) Retries for External Integrations (most critical)

Failures in external calls (through the Integration Hub) are handled with bounded retries, applied only to **transient** conditions such as:

- timeouts / connectivity issues
- temporary unavailability
- 5xx responses

Retry policy

- exponential backoff with jitter

- maximum retry attempts
- maximum retry window per processing step

B) Retries for Internal Processing

Transient internal errors (e.g., temporary DB contention) may be retried within bounded limits. Persistent failures should not loop indefinitely.

Failure classification

- **Transient** → automatic retry

- **Business/validation failure** → no retry; inquiry transitions to a final non-success outcome with reason
- **Persistent technical failure** → stop retry after limits; mark inquiry as "Failed – Requires Support Review" (no admin escalation feature; operational follow-up only)

Dead-Letter Handling and Operational Recovery

When a processing job exceeds retry limits, it is treated as **dead-lettered** logically:

- The inquiry is marked with a terminal technical failure state and a failure reason reference.
- Technical details (error category, correlation ID, last exception) are recorded in logging/tracing.
- The case becomes visible for support follow-up via operational monitoring (without introducing an admin escalation feature).

Idempotency and Duplicate Protection

To avoid duplicate processing and double-charging scenarios:

- Inquiry submission endpoints (especially agency/system integrations) should support **idempotency keys**.
- Processing jobs must be idempotent per inquiry step (safe re-run without duplicating outcomes).
- Payment confirmation callbacks from the external payment gateway must be idempotent (same callback must not apply wallet/payment effects twice).

4.2.6 Rules & Reference Data (Configuration)

Purpose

The **Rules & Reference Data** module provides a centralized, governed repository for all **configurable parameters** and **reference lists** required by the solution. It enables the solution to apply policies (eligibility, fees, priorities, integrations) consistently without embedding such rules directly in user interfaces or processing logic.

This module is the authoritative source for configuration used by Inquiry Management, Clearance Processing, and administrative functions in the Admin Control Portal.

Primary Users

- **Administrative Users** (via Admin Control Portal) for configuration and governance
- **Downstream Modules**
  - Inquiry Management
  - Clearance Processing
  - Integration Hub (indirectly, via endpoint/entity configuration)
  - Reporting & Dashboards (for reference interpretation)

Key Responsibilities

- Maintain configurable business rules and parameters used by the solution.
- Manage reference datasets (e.g., nationalities, countries, document types, statuses).
- Maintain SLA-related configuration settings (where required).
- Maintain priority policy configuration (Express/Standard/Batch enablement and constraints).
- Manage fees/pricing configuration (including express fees if applicable).
- Manage integration configuration (external entity profiles, endpoints, routing policies).
- Support controlled updates with auditability and safe rollout.

Sub-Systems and Functional Areas

Policy & Eligibility Configuration

- Eligibility conditions per nationality/category as defined by policy
- Mandatory data/fields rules (what is required at submission)
- Disallowed combinations (where applicable)

SLA & Priority Policy Configuration

- SLA targets configuration (per priority level if applicable)
- Priority enablement rules:

  - who can select Express vs Standard vs Batch
  - any constraints by user type (traveler/agency)
- Batch fairness policy parameters (e.g., worker cap configuration value if stored as config)

Fees & Pricing Configuration

- Fees per service type / priority (if pricing differs)
- Currency and payment-related policy parameters
- Fee change governance (effective dates if needed)

Reference Data Management

- Master lookup lists used across the solution:
  - nationalities, countries, document categories
  - statuses, reason codes, decision codes
- Consistent interpretation for UI, APIs, and reporting

Integration Configuration Repository

- External entity profiles (gov/non-gov/payment gateway identifiers)
- Endpoints, protocol parameters, and routing configuration (high-level)
- Timeout and retry policy parameters (policy values used by Integration Hub)

Versioning and Governance

- Maintain versioned configuration sets or effective date snapshots (policy-driven)
- Record configuration changes with:
  - who changed what
  - when it was changed
  - why / change reference (if required)

4.2.7 QR Verification & Document Authenticity

Purpose

The **QR Verification & Document Authenticity** feature provides a controlled mechanism to validate issued clearance approvals. It ensures that each issued document contains a secure QR reference enabling authorized users to verify approval validity and retrieve associated approval details according to access policy.

The feature is implemented using:

- Document Issuance for generating documents and embedding QR references.
- A Verification Endpoint (UI/API) for performing verification by authorized users.
- Identity Hub for authentication and authorization.
- Audit and Observability for traceability.

Primary Users

- **Authorized Verifiers** (role-based), such as:
  - Administrative users
  - Authorized agency users (if enabled by policy)

Key Responsibilities

- Generate and embed a non-predictable QR reference in issued documents.
- Provide an authorized verification capability that validates document status and approval validity.
- Ensure verification output is consistent and policy-compliant.
- Record verification and download activities for audit and troubleshooting.

Feature Components and Functional Areas

A) QR Reference Generation (Document Issuance)

- Generate a cryptographically strong, non-guessable QR token/reference.
- Associate QR reference with the inquiry approval and expiry window.
- Embed QR reference in the generated PDF.

B) Authorized Verification Endpoint (Presentation)

- Require authentication and authorization via Identity Hub.
- Accept QR reference/token and validate:
  - document exists (logically)
  - approval is valid and not expired
- Return verification result in a controlled view (policy-driven content).

C) On-the-Fly Document Generation and Download

- Generate document upon request using current template and current approved data.
- Enforce authorization (only the entitled traveler/agency can download).
- Support controlled bulk downloads for agencies (where applicable).

D) Audit, Logging, and Monitoring

- Record:
  - document download events
  - verification events (who verified, when, outcome)
- Emit telemetry and error indicators for operational monitoring.

Key Dependencies

- **Identity Hub** (authentication / authorization)
- **Inquiry Management** (approval state, expiry, status)
- **Document Issuance** (document generation, QR embedding)
- **Audit Trail** (verification and download audit records)
- **Logging & Tracing** (correlation and diagnostics)
- **Monitoring & Telemetry** (availability, latency, failures)

4.2.8 Dashboards & Reporting

Purpose

The **Dashboards & Reporting** component provides near real-time dashboards and reporting capabilities to support both **operational monitoring** and **business decision-making**. It enables authorized users to view inquiry volumes, status distributions, SLA indicators, and trends, with the ability to **export** filtered datasets and reports.

The component supports two main dashboard perspectives:

- **Operational / SLA Monitoring views** for Gahez teams through the Admin Control Portal.
- **Business Insights views**, where Web Portal users see **tenant-scoped dashboards**, while Gahez users can access **global, cross-tenant dashboards**.

Near real-time dashboards operate with a **minutes-level delay**.

Primary Users

- **Gahez Team (Admin Control Portal)**
  - Operations and support teams
  - Decision makers and management
- **Tenant Users (Web Portal)**
  - Agencies accessing tenant-scoped dashboards and reports

Key Responsibilities

- Provide near real-time dashboards with minutes-level delay.

- Support **multi-tenant reporting** with strict tenant isolation for Web Portal users.
- Provide **global reporting** and oversight dashboards for Gahez users.
- Enable export of dashboards and reports (CSV / Excel, and optionally PDF).
- Ensure reporting workloads do not impact core transactional processing.
- Enforce role-based and tenant-based access control across all dashboards and exports.

Sub-Systems and Functional Areas

A) Operational / SLA Monitoring Views (Admin Control)

- SLA compliance indicators by priority level (Express / Standard / Batch)
- Processing throughput, average lead times, and aging buckets
- Backlog and queue health indicators
- Integration and payment-related failure indicators
- Operational drill-down with filtering and export capabilities

B) Business Insights Views (Multi-Tenant)

- Tenant-scoped dashboards (Web Portal)**:**
    - Inquiry volumes and outcomes for the tenant
    - SLA indicators for tenant submissions
    - Batch submission progress and completion metrics
    - Export of tenant-scoped datasets
- Global dashboards (Admin Control)**:**
    - Cross-tenant KPIs and trends
    - Approval vs rejection ratios
    - Breakdown by nationality, category, and priority
    - SLA compliance trends across tenants
    - Export of global and filtered results

C) Reporting Data Model, Refresh, and Export

- Use of **read-optimized reporting models** (views or read tables) to support fast filtering and aggregation.
- Near real-time refresh strategy with minutes-level delay, driven by background processing.
- Scheduled aggregation and refresh jobs using **Hangfire** where required.
- Server-side enforcement of:
    - role-based access control
    - tenant-based data filtering
- Export generation that respects access policies and avoids impact on transactional workloads.

- Auditability of export actions and access to sensitive reporting views.

Figure 7 Dashboards & Reporting (Operational + Business Views)

Key Dependencies

- **Identity Hub** (roles, permissions, tenant claims)
- **Inquiry Management / Clearance Processing** (source facts and timestamps)
- **Operational Data Store** and **Reporting Read Models**
- **Hangfire** (aggregation, refresh, and export jobs)
- **Audit Trail** (export and access tracking)
- **Logging & Tracing** (diagnostics and correlation)
- **Monitoring & Telemetry** (availability and performance of dashboards)

4.2.9 Integration Hub

Purpose

The **Integration Hub** provides a unified and secure integration layer between the solution and external entities (governmental and non-governmental). It decouples core processing modules from external system variations by standardizing how requests are routed, secured, tracked, retried, and correlated.

The Integration Hub reduces integration complexity, supports future extensibility, and enables consistent observability across all external communications.

Primary Users

- **Clearance Processing** (primary consumer for external validations)
- **Inquiry Management** (indirect consumer, if submission requires external checks)
- **Administrative Users / Gahez Operations Team** (via Admin Control for monitoring integration health and failures)

Key Responsibilities

- Provide a **single integration entry point** for all outbound/inbound external communications

- Standardize request/response handling (validation, mapping, correlation, and error handling).
- Route requests to the correct external entity based on configuration and policy.
- Enforce integration security (authentication, authorization, certificates, encryption).
- Implement resilient communication patterns:
    - timeouts
    - bounded retries with backoff
    - circuit breaker behavior (conceptual at HLD level)
- Provide consistent integration logging, tracing, and metrics.
- Persist integration interaction metadata needed for auditability and troubleshooting.

Sub-Systems and Functional Areas

A) Connector Adapters (Per External Entity / Protocol)

- Governmental connectors (as per configured endpoints)
- Non-governmental connectors (supporting validations)
- Each connector encapsulates:
    - protocol details (REST/SOAP/etc.)
    - authentication mechanism
    - payload mapping to/from canonical format
    - error normalization

B) Canonical Integration Contracts (Normalization Layer)

- Define canonical request/response contracts used internally by the solution.
- Map external variations into internal standardized structures.
- Ensure internal modules (Clearance Processing) do not depend on external schemas.

C) Routing and Policy Engine

- Select target external system(s) based on:
    - nationality/category rules
    - inquiry type
    - configured routing policies
- Support multi-target orchestration (send to multiple entities and aggregate results).

D) Resilience and Fault Handling

- Timeouts per connector
- Bounded retries with exponential backoff + jitter (transient failures only)
- Failure classification:
  - transient → retry
  - persistent/contract mismatch → fail fast
- Controlled degradation (e.g., mark inquiry as failed or incomplete according to policy)

Note: Retry scheduling and job execution align with the Hangfire-based processing model defined earlier. The Integration Hub defines per-connector retry policies; execution occurs within background processing jobs.

E) Correlation, Traceability, and Auditing

- Generate and propagate correlation IDs across:
  - internal modules
  - external requests
  - external responses
- Persist integration interaction metadata:
  - who/what triggered the call
  - timestamp
  - external target
  - outcome (success/failure)
- Provide references usable by:
  - audit trail
  - logging & tracing
  - operations dashboard drill-down

F) Integration Health and Monitoring

- Provide health indicators per connector:
  - availability
  - latency
  - error rate
  - timeout count
- Surface indicators to dashboards/operations console.

Key Dependencies

- **Rules & Reference Data**
  - External entity profiles, endpoints, routing policies, timeouts/retry parameters
- **Clearance Processing**
  - Triggers validations and consumes normalized results

- **Logging & Tracing**
  - Distributed correlation and diagnostic events
- **Audit Trail**
  - Integration-related auditable events (high-level)
- **Monitoring & Telemetry**
  - Metrics, health checks, alerting inputs
- **Data Layer**
  - Storage of integration interaction metadata and correlation references

Integration Techniques with External Entities (HLD-Level)

1) Integration Interaction Styles

The solution supports multiple interaction styles depending on the external entity capabilities and the nature of the operation:

- **Synchronous Request/Response (Real-time)**
  - Used for lightweight validations where immediate response is required.
  - Typical protocols: REST/JSON, SOAP/XML.
  - Governed by strict timeouts and bounded retry policies.
- **Asynchronous Integration (Recommended for long-running or unreliable endpoints)**
  - Used when the external entity response is delayed or requires multi-step processing.
  - Implemented using background processing (Hangfire) and polling/callback patterns.
  - Suitable for scenarios where multiple entities must respond before final decision.
- **Hybrid Orchestration**
  - A single inquiry may require a mix of synchronous validations and asynchronous completion.
  - Clearance Processing orchestrates the flow; Integration Hub normalizes and tracks all steps.

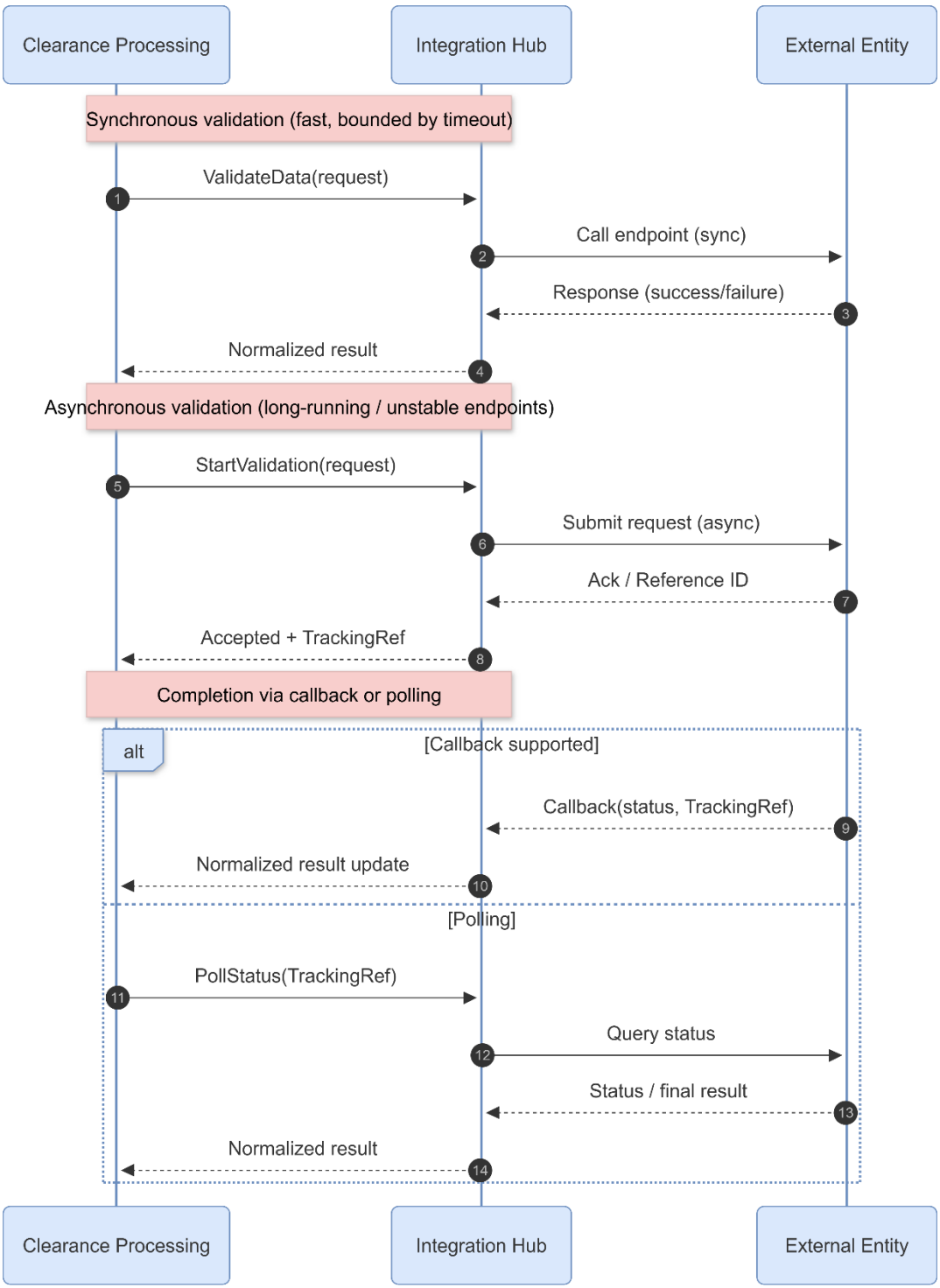

Figure 8 Integration Patterns (Sync vs Async) Sequence

2) Supported Integration Patterns

- **Adapter / Facade Pattern**
  - Each external entity is integrated through a dedicated adapter that encapsulates protocol, authentication, and mapping.

- **Canonical Data Contract Pattern**
  - Internal modules communicate using a stable internal contract; adapters map external schemas to canonical models.
- **Gateway / Policy-Based Routing**
  - Integration routing is driven by configuration (entity profile, nationality/category, inquiry type).
- **Idempotency Pattern**
  - Outbound requests and inbound callbacks use idempotency keys to prevent duplicate processing.

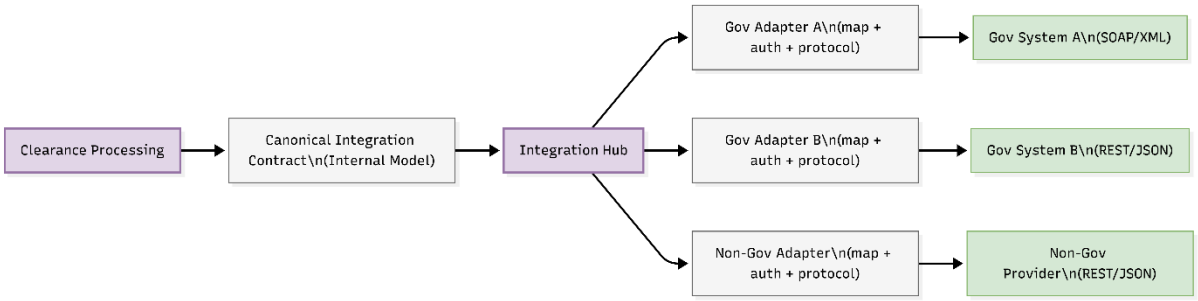

Figure 9 Canonical Contract + Adapters

While below diagram Illustrates how the Integration Hub protects the solution from unstable or failing external systems using timeouts, retries, and controlled failure handling.

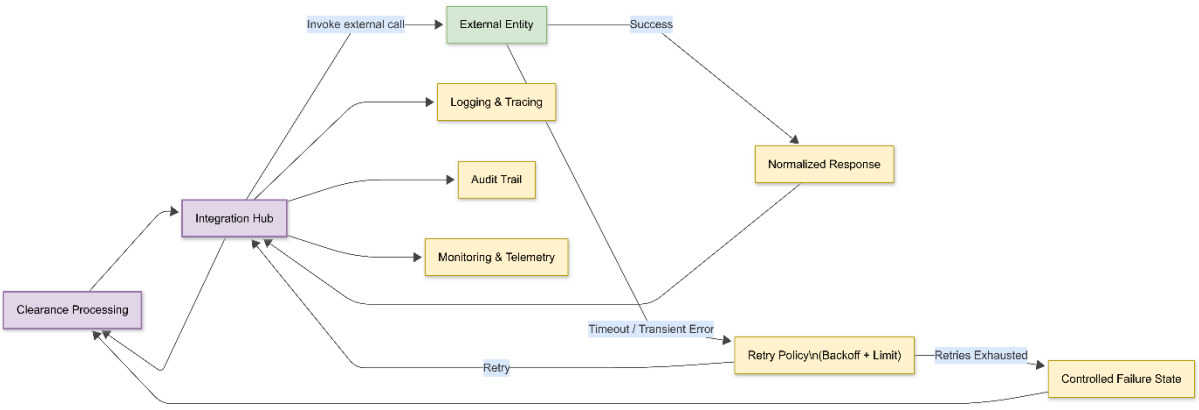

Figure 10 Integration Resilience Control Loop

3) Request Correlation and Traceability

- A **Correlation ID** is generated per inquiry and propagated across:
  - internal modules
  - outbound external calls
  - inbound callbacks (if applicable)

- All integration interactions are traceable via:
  - correlation ID
  - external request reference (if provided by external entity)
  - timestamps and outcome codes

4) Resilience Techniques (Non-Functional Integration Guarantees)

- **Timeouts per adapter**
  - Prevents thread starvation and protects SLA.
- **Bounded retries with exponential backoff + jitter**
  - Applied only to transient failures (timeouts, 5xx, temporary connectivity).
- **Circuit breaker behavior (conceptual)**
  - If repeated failures occur, the solution temporarily blocks calls to the failing connector to protect overall system health.
- **Dead-letter / failure state handling**
  - When retries are exhausted, inquiry processing transitions to a controlled failure state for operational follow-up.

5) Security Techniques for External Integrations

Depending on the external entity capabilities, the solution supports:

- **Mutual TLS (mTLS)** where required (certificate-based authentication).
- **OAuth2 / JWT** when supported by the external entity.
- **API Key / HMAC signing** for simpler integrations (least preferred, controlled by policy).
- **IP allowlisting and firewall rules** (in coordination with hosting standards).
- **Payload encryption / signing** where required by policy.
- **Input validation and schema enforcement** on all inbound integration payloads.

6) Data Exchange Formats and Compatibility

- Primary formats:
  - **JSON** for modern REST APIs
  - **XML** for SOAP or legacy systems
- The Integration Hub handles:
  - schema validation (where feasible)
  - strict contract versioning
  - backward compatibility at adapter level

7) Integration Modes (Outbound vs Inbound)

- **Outbound integrations**
  - The solution initiates calls to external entities for validations and data enrichment.
- **Inbound callbacks/webhooks (if supported)**
  - External entities may push decisions or status updates to the solution.
  - Inbound requests are authenticated, validated, idempotent, and correlated.

4.2.10 Identity Hub (Access Management)

Purpose

The **Identity Hub** provides centralized authentication and authorization capabilities for all solution channels (Web Portal, Admin Control Portal, Partner/System APIs). It enforces secure access, role-based permissions, and tenant-scoped visibility, ensuring that each user or system client only accesses allowed resources.

Identity Hub is a shared supportive module used across the solution and is essential for implementing **multi-tenant isolation**, **role-based dashboards**, and secured operational features such as QR verification and export.

Primary Users

- **Travelers** (Web Portal authentication)
- **Agency Users** (Web Portal authentication)
- **Gahez Team Users** (Admin Control authentication)
- **System Clients / Integrators** (Partner/System APIs authentication)
- **Internal Modules** requiring authorization decisions (reporting/export, verification, admin functions)

Key Responsibilities

- Authenticate users and system clients securely.
- Enforce role-based access control (RBAC) across all portals and APIs.
- Enforce tenant-based access control using tenant claims (e.g., AgencyId/TenantId).
- Issue and validate access tokens for APIs and portals.
- Support secure session management and token lifecycle controls.
- Provide identity administration functions required by the solution (user provisioning, role assignment) based on scope.

Sub-Systems and Functional Areas

Authentication Services

- User login / logout for portals
- Token issuance for APIs (JWT or equivalent)
- Token validation and session management
- Multi-factor authentication support (if required by policy)

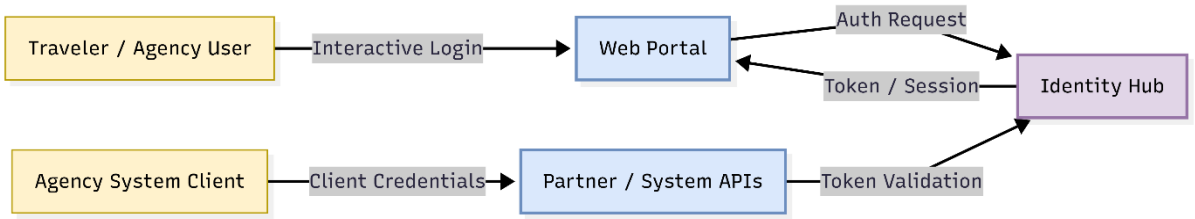

Figure 11 Authentication Models by Channel

Authorization Services (RBAC + Policies)

- Role-based permissions for:
  - traveler operations
  - agency operations
  - Gahez admin operations
  - verifier roles (for QR verification if enabled)
- Policy-based authorization checks for sensitive actions:
  - exports
  - bulk downloads
  - cross-tenant views

Tenant Claim Management (Multi-Tenant Enforcement)

- Assign tenant identifier claims for tenant users (e.g., AgencyId)
- Ensure all tenant-scoped services consume tenant claims server-side
- Prevent cross-tenant access by design

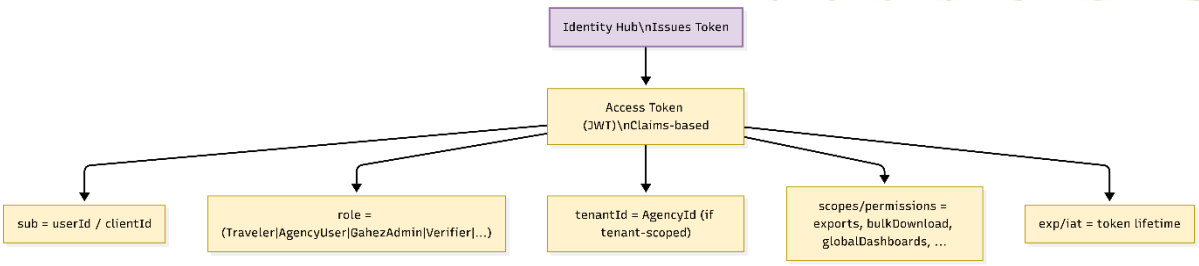

Figure 12 Token and Claims Model (JWT Claims)

System Client Authentication (API Consumers)

- Client credential management (for agency systems / service providers)
- API key / OAuth client credentials / certificate-based auth (based on integration policy)

- Rate limits and usage governance hooks (if required)

Identity Administration (Scope-Driven)

- Basic user provisioning and deactivation
- Role assignment and revocation
- Optional integration with enterprise directory (if applicable)

Hybrid Authentication & Authorization Flow (Web Portal & Application Layer)

The solution adopts a **hybrid security flow** that combines modern SPA authentication with strict server-side authorization. The Web Portal (Angular SPA) authenticates users via the Identity Hub using **OIDC Authorization Code Flow with PKCE**. After authentication, the portal invokes backend APIs using short-lived access tokens. The Application Layer validates tokens and enforces **RBAC and tenant-based access control** server-side to prevent unauthorized access and ensure multi-tenant isolation.

Key security characteristics:

- Tokens are **short-lived** and validated by the backend on every request.
- Authorization decisions are enforced at the API boundary (Policy Enforcement Point).
- Tenant isolation is enforced via **tenant claims** and server-side filtering.
- Refresh/token renewal is handled securely through Identity Hub mechanisms (policy-driven).

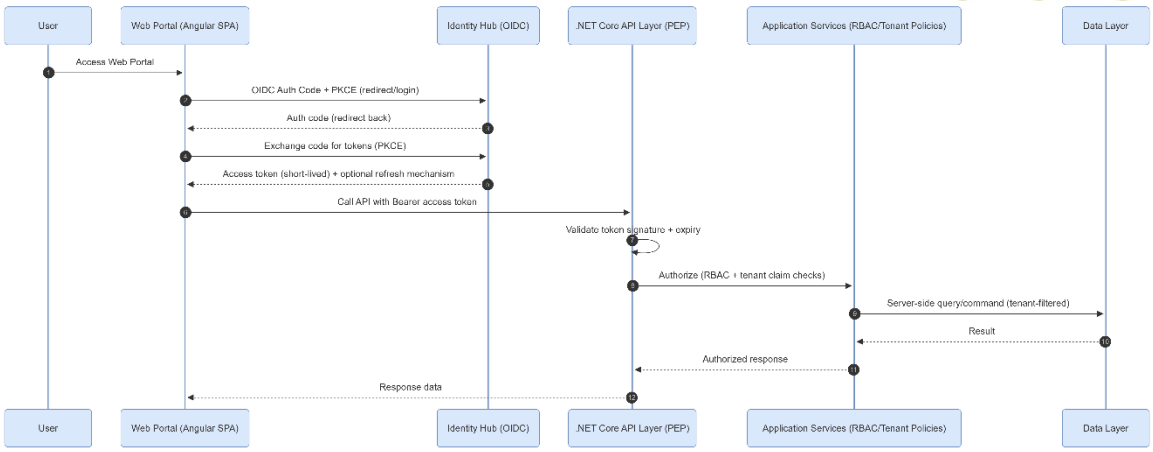

Figure 13 Hybrid Authentication & Authorization Flow

4.2.11 Notification Manager

Purpose

The **Notification Manager** provides a centralized mechanism for generating and sending outbound notifications to end users and partners based on key events in the solution (e.g., inquiry submitted, status updated, decision finalized, document available). In the initial release, notifications are delivered through **Email only**, using **Gahez Exchange email** as the outbound channel.

The module decouples notification logic from core processing modules, ensuring consistent formatting, traceability, and controlled delivery.

Primary Users

- **Travelers** (receive inquiry status and outcomes)
- **Agency Users** (receive batch status and outcomes)
- **Gahez Team** (optional operational notifications if enabled later)

Key Responsibilities

- Send event-driven email notifications triggered by core modules.
- Provide standardized notification templates and message structure.
- Ensure reliable sending with basic retry and failure handling.
- Record notification activity for traceability (sent/failed, timestamp, recipient reference).
- Support controlled throttling to avoid email bursts (especially for batch scenarios).

Sub-Systems and Functional Areas

A) Notification Triggers (Event Sources)

Notifications are triggered by key system events, such as:

- Inquiry submitted (optional)
- Payment confirmed (if applicable)
- Decision finalized (approved/rejected)
- Document available for download
- Batch completion summary (for agencies)

Triggers should be implemented through internal events or service calls to keep business modules decoupled.

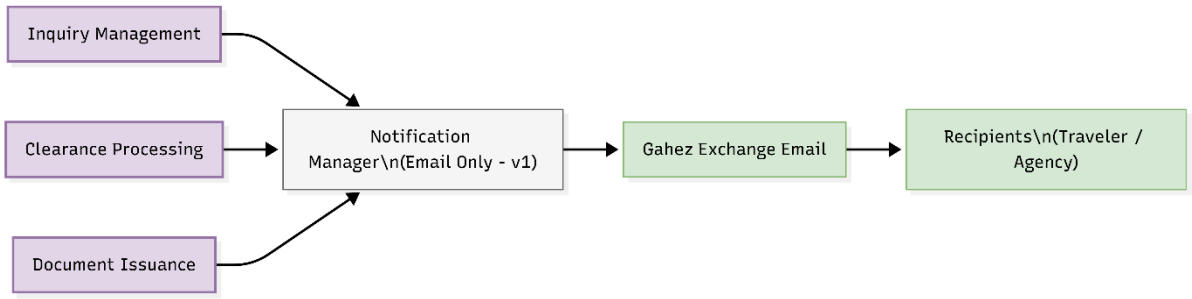

Figure 14 Notification Manager (Email-Only v1)

B) Template and Content Management (Email)

- Centralized email templates (subject + body).
- Localization readiness (Arabic / English) if needed.
- Parameter binding from inquiry data (e.g., inquiry reference, status, expiry date).
- Standard footer and disclaimer (policy-driven).

In v1, templates can be stored as simple HTML files or database records (implementation choice deferred to Build View).

C) Delivery Channel (Email via Gahez Exchange)

- Outbound email sending uses **Gahez Exchange** mail service.
- Each email is sent from an official sender identity (e.g., no-reply or service mailbox).
- Email delivery errors are captured and classified for operational follow-up.

D) Reliability, Retry, and Idempotency (Lightweight v1)

- Email send attempts use bounded retries (e.g., transient connectivity issues).
- Idempotency is applied to avoid duplicate messages for the same event:
  - unique key = (NotificationType + InquiryId + EventVersion)
- Failures are recorded and surfaced to operations dashboards (basic visibility).

E) Auditability and Observability

- For each notification:
  - recipient reference
  - type (decision, document available, etc.)
  - timestamp
  - outcome (sent/failed)
  - correlation ID (linked to inquiry processing trace)

This supports troubleshooting and operational reporting.

4.2.12 Audit Trail

Purpose

The **Audit Trail** component captures and preserves **significant business and security events** across the solution in a structured, queryable, and controlled manner. It provides traceability for compliance, investigations, incident response, and operational governance by answering: **who did what, when, on which object, and with what outcome**.

Audit Trail is distinct from technical logging; it records **business/security intent and outcomes**, not low-level diagnostics.

Primary Users

- **Gahez Operations / Security Team** (investigations, compliance, governance)
- **System Administrators** (review of sensitive activities)
- **Support Team** (traceability for user inquiries and disputes)
- **Auditors** (internal/external, if applicable)

Key Responsibilities

- Capture significant business and security events from portals, APIs, and background workers.
- Store audit events in a consistent schema with clear semantics.
- Enforce auditability for sensitive actions (exports, configuration changes, verification).
- Support search and reporting (time range, user, tenant, inquiry, event type).
- Apply retention and access control policies for audit data.

Audit Data Model (Fields and Meaning)

Field	Meaning / Notes
AuditEventId	Unique identifier for the audit record (GUID).
CorrelationId	Links multiple audit events within the same business flow (e.g., inquiry lifecycle).
EventType	Standardized event name (e.g., InquirySubmitted, DecisionFinalized, ExportGenerated).
EventTimestampUtc	When the audited event occurred (UTC).
ComponentId	Module emitting the event (InquiryManagement, IntegrationHub, Reporting, AdminControl, etc.).
Channel	Source channel (WebPortal, AdminControl, PartnerAPI, BackgroundWorker).

TenantId (nullable)	Tenant identifier (e.g., AgencyId). Null for global/system events.
UserId (nullable)	Authenticated user initiating the action (for user-driven events).
ClientId (nullable)	System client identifier for API client-credential scenarios.
SessionId (nullable)	Session identifier if applicable for portal sessions.
Action / Operation	Human-readable operation label (e.g., ExportReport, VerifyQR, ConfigureRules).
Outcome	Success / Failure (optionally Partial/Timeout for integration-related high-level events).
OutcomeReasonCode (optional)	Normalized reason code (ValidationFailed, ExternalTimeout, Unauthorized, etc.).
RelatedObjectType	Domain object type (Inquiry, Document, Payment, RuleConfig, User, IntegrationRequest).
RelatedObjectId	Domain object identifier (e.g., InquiryId).
Description	Short business description (avoid sensitive data).
ExtraDetailsJson (optional)	Structured JSON for non-sensitive details needed for investigations (avoid PII).
DataClassification (optional)	Classification label to guide masking/visibility (Internal/Confidential).

Important: we do not store sensitive PII in ExtraDetailsJson. Prefer identifiers and masked values.

Audited Event Categories (Examples)

Business Events

- Inquiry submitted / updated
- Payment confirmed / failed (high-level reference only)
- Decision finalized (approved/rejected)
- Document downloaded (high-level record)
- Batch completion summary (for tenant scope)

Security & Admin Events

- Role/permission changes
- Configuration changes (rules, fees, integrations)
- Report export actions
- Authorized QR verification actions (if enabled)
- Administrative access to global/cross-tenant dashboards (if policy requires)

Integration Events (High-Level)

- External request initiated (optional)
- External response received (optional)
- Retry exhausted / timeout (captured as outcome + reason code, correlated to inquiry)

Techniques and Methodologies

A) Audit Implementation Technology

The solution adopts **Audit.NET** as the auditing framework to capture structured audit events and persist them to the audit store.

B) Storage Approach (Direct Write to SQL Server)

Audit events are persisted directly into a dedicated **SQL Server Audit Trail store**. The store is treated as **append-only** from an application perspective (no updates to historical audit records).

If correction is needed, a new audit event is created that references the original event.

C) Reliability Strategy (Criticality-Based Handling)

Because the audit store and operational store use the same database technology (SQL Server), a separate buffering layer is not introduced in v1. Instead, reliability is ensured through **event criticality rules**:

- **Critical audit events (must be recorded):**
  configuration changes, role/permission changes, exports, verification actions, decision finalization, and other privileged actions.
  Rule: if audit persistence fails, the operation is not completed and returns an error.
- **Non-critical audit events (best-effort):**
  high-volume informational events (e.g., optional view/download events where appropriate).
  Rule: if audit persistence fails, the business operation may continue, while a technical log entry and telemetry alert are emitted for operational follow-up.

D) Access Control and Data Protection

- Audit store access is restricted to privileged roles only.
- Audit query endpoints are read-only and protected through Identity Hub (RBAC + tenant policies).
- Sensitive details are excluded or masked according to data classification policy.

E) Correlation with Logs/Traces

Audit events include a **CorrelationId** shared with technical logs/traces (Serilog + OpenTelemetry), enabling end-to-end investigation without mixing audit intent with technical diagnostics.

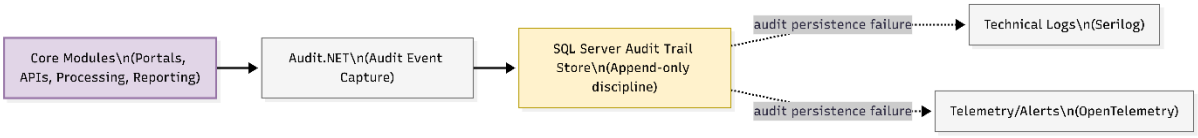

Figure 15 Audit trail

4.2.13 Logging & Tracing

Purpose

The **Logging & Tracing** component provides end-to-end technical observability for the solution by capturing structured logs, distributed traces, and correlation data across all channels and modules. It enables effective troubleshooting, performance analysis, and root-cause investigation without overlapping with business-level auditing responsibilities.

This component focuses on **how the system behaves**, not **what business action occurred** (which is handled by the Audit Trail).

Primary Users

- **DevOps / Operations Team**
- **Support & Engineering Teams**
- **Gahez Platform Administrators** (technical oversight)

Key Responsibilities

- Capture structured technical logs across portals, APIs, background jobs, and integrations.
- Provide distributed tracing across synchronous and asynchronous flows.
- Propagate and enforce correlation IDs across all layers.
- Capture performance and error diagnostics without exposing sensitive data.
- Enable efficient filtering and analysis of logs and traces.

Logging & Tracing Scope Definition

Aspect	Responsibility

Logs	Technical events, errors, warnings, diagnostics
Traces	End-to-end request flow across components
Metrics	Latency, error rate, throughput (via telemetry)
Audit	Out of scope (handled by Audit Trail)

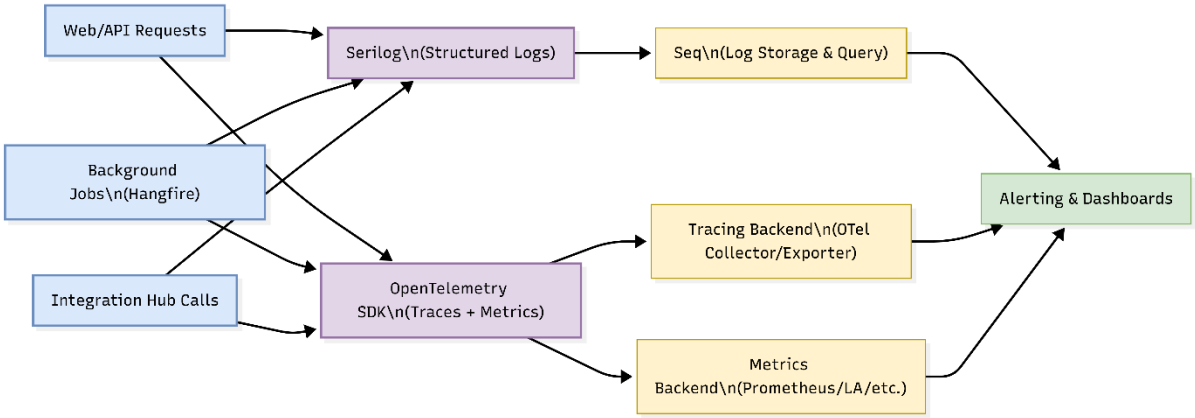

Figure 16 logging and tracing

Techniques and Methodologies

A) Structured Logging (Serilog)

The solution uses **Serilog** to emit structured, machine-readable logs across all components.

Key principles:

- Logs are structured (key/value), not free-text.
- Log levels are used consistently (Information, Warning, Error, Critical).
- Sensitive data (PII, credentials, tokens) is never logged.
- Logs include contextual enrichment:
  - CorrelationId
  - Component/Module name
  - Environment
  - TenantId (where applicable)

Typical log events:

- API request/response failures
- Background job failures (Hangfire)
- Integration errors and retries
- Validation and processing exceptions

B) Distributed Tracing (OpenTelemetry)

The solution adopts **OpenTelemetry (OTel)** for distributed tracing to track request flows across:

- Web Portal
- Admin Control
- Partner/System APIs
- Application services
- Integration Hub
- Background jobs

Tracing capabilities include:

- Trace and span generation per request
- Span enrichment with correlation and context data
- Visibility into latency contributors (internal vs external calls)

C) Correlation ID Propagation

A **CorrelationId** is generated at the system boundary and propagated across:

- HTTP requests
- Background jobs
- Integration calls
- Logs, traces, and audit events

This enables:

- Single-click tracing from error → trace → logs → audit record
- Consistent investigation across modules

D) Error Classification & Signal Quality

Errors are categorized to improve observability quality:

- **Application errors** (validation, business rule failures)
- **Integration errors** (timeouts, retries exhausted)
- **Infrastructure errors** (DB connectivity, service availability)

Only actionable errors are promoted to alerts.

E) Storage and Analysis (Abstracted)

At HLD level, the solution supports exporting logs and traces to:

- Centralized log storage (e.g., log analytics platform)

- Distributed tracing backend
- Monitoring dashboards and alerting tools

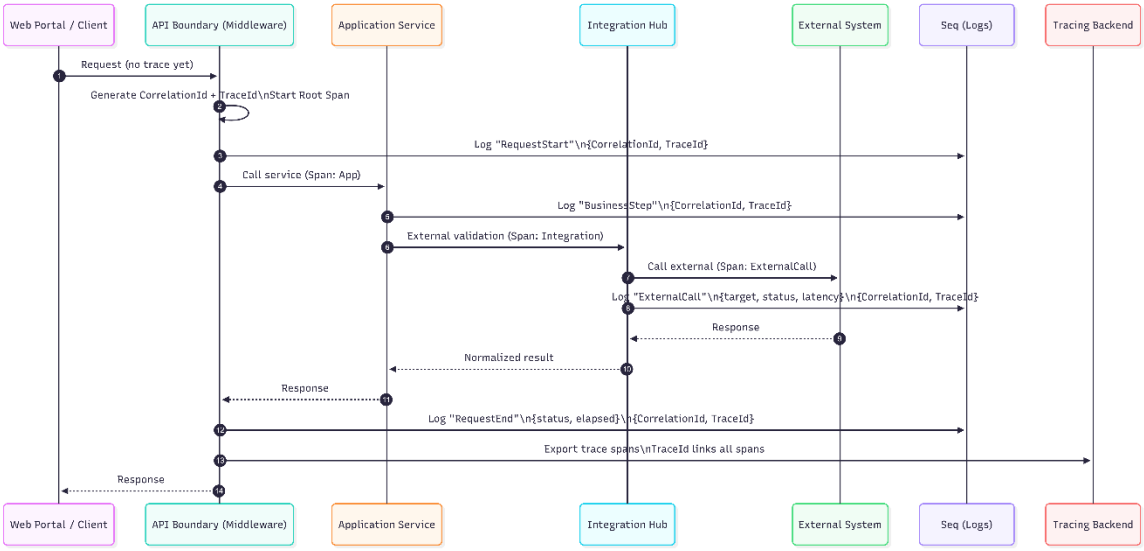

Figure 17 logging sequence diagram

4.2.14 Health Check & Service Status

Purpose

The **Health Check & Service Status** component provides a standardized mechanism to verify the runtime health of the solution's modules and their critical dependencies. It supports both **operational monitoring** and **automated environment readiness validation**, enabling early detection of outages, degraded dependencies, and configuration issues.

This component is not a functional feature for end users; it is an operational capability used by the Gahez team and platform operations.

Primary Users

- **Gahez Operations / DevOps Team**
- **Support Team**
- **Automated Monitoring Tools** (polling health endpoints)

Key Responsibilities

- Expose health endpoints for runtime liveness and readiness checks.
- Provide dependency health checks for:

    - SQL Server connectivity
    - Integration endpoints reachability (optional "synthetic" checks)
    - Email (Exchange) connectivity (basic)
    - Hangfire infrastructure (job storage availability)
- Support differentiated health states:
    - **Healthy**
    - **Degraded** (partial dependency issues)
    - **Unhealthy**
- Provide a summary health model consumable by monitoring dashboards and alerting.

Sub-Systems and Functional Areas

A) Liveness Checks

- Confirms the service/process is running and able to respond.
- Minimal logic; no deep dependency verification.

B) Readiness Checks

- Confirms the module is ready to serve requests.
- Includes critical dependency verification, such as:
    - database reachable
    - required configuration loaded
    - background processing available (if required)

C) Dependency Health Checks (Policy Driven)

- **Database health**: connection + minimal query
- **Hangfire health**: storage reachable + workers active (high-level)
- **Email channel health**: Exchange reachable (basic connectivity)
- **External integrations health**: endpoint reachability + SLA signals (optional)

External dependencies may be reported as **Degraded** rather than failing full readiness, based on business tolerance and SLA policies.

D) Health Status Aggregation

- A single aggregated health status per environment/module.
- Includes breakdown by dependency for troubleshooting.

E) Secure Access to Health Endpoints

- Health endpoints are protected based on deployment context:

- internal network access only, or
- authenticated access for privileged roles

Key Dependencies

- **SQL Server** (connectivity and minimal query checks)
- **Hangfire job storage** (availability signal)
- **Integration Hub** (optional synthetic checks)
- **Notification Manager** (Exchange connectivity check)
- **Monitoring & Telemetry** (publish health status metrics)

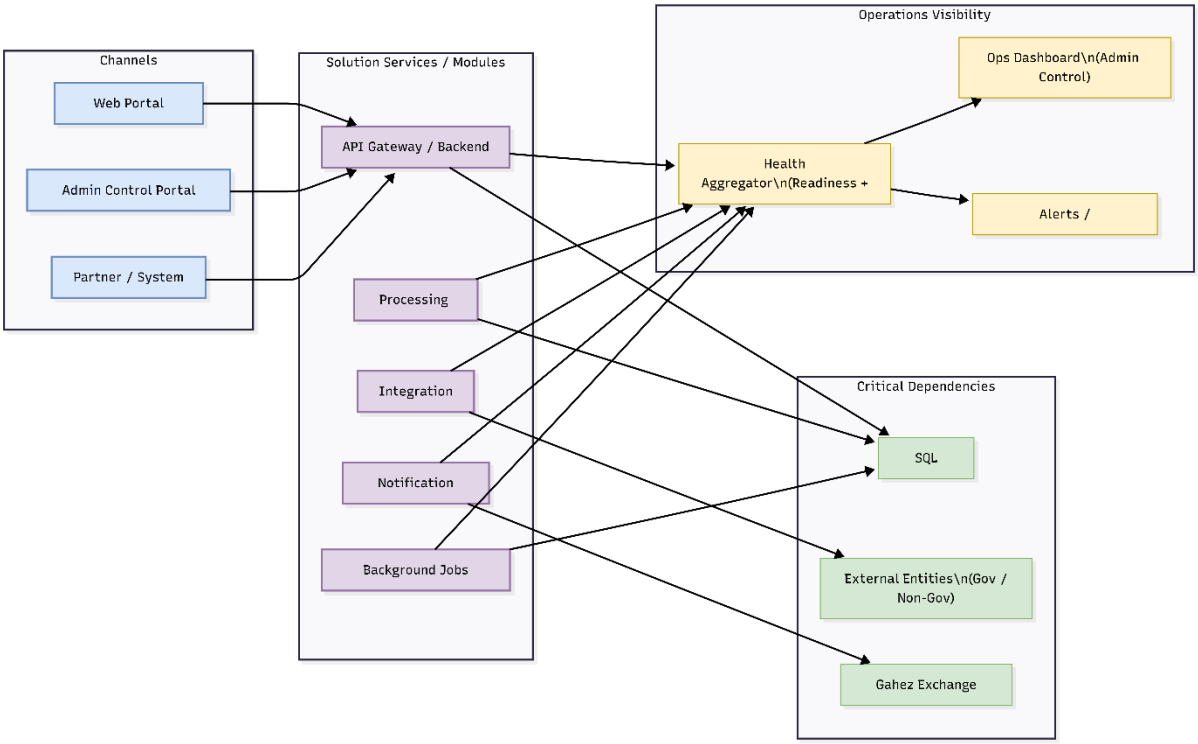

Figure 18 Aggregated health check

The solution exposes standard liveness/readiness health indicators per service/module. A **Health Aggregator** consolidates health states (Healthy/Degraded/Unhealthy) across services and dependencies and surfaces them to the **Admin Control operations dashboard**, with optional alerting for critical degradations.

4.3 Build View

4.3.1 Implementation Pattern

Earlier sections describe architectural principles and techniques. This section defines the **concrete implementation patterns** (codebase structure, module boundaries, layering rules, dependency rules) that will be applied to realize the target architecture.

Implementation Approach Overview

The solution will be implemented as a **modular monolith** using **Clean Architecture** principles, with clear module boundaries aligned to the functional blocks. The codebase is structured to keep domain and application logic independent from infrastructure concerns (database, external integrations, email, background processing).

Where beneficial, the solution adopts a **CQRS-light** approach: commands and queries are separated at the application layer, and reporting is supported through dedicated read models.

A) Modular Monolith Packaging (Solution-Level)

The solution is organized into functional modules that reflect the architecture blocks. Each module is responsible for its own application and domain logic and depends on shared cross-cutting services only through defined abstractions.

Indicative modules (aligned to 4.2 Functional Blocks):

- Presentation:
  - Web Portal (Angular)
  - Admin Control Portal (Angular)
  - Partner/System APIs (.NET)
- Application/Processing:
  - Inquiry Management
  - Clearance Processing
  - Document Issuance
  - Rules & Reference Data
  - Dashboards & Reporting (Query services + exports)
- Integration:
  - Integration Hub (adapters, routing, resilience policies)
- Support/Operations:
  - Identity Hub
  - Notification Manager (email v1)
  - Audit Trail (Audit.NET)
  - Logging & Tracing (Serilog + OpenTelemetry)
  - Health Checks / Service Status

Module communication (within the monolith):

- Primary: in-process service calls through interfaces (application layer contracts)

- Optional: internal domain/application events for decoupling (e.g., "DecisionFinalized" triggers notification + audit)

B) Clean Architecture Layering (Per Module)

Each module follows Clean Architecture layering:

- **Presentation Layer**
  - Controllers/API endpoints
  - Request validation (boundary-level)
  - Authentication/authorization enforcement (policy enforcement point)
- **Application Layer**
  - Use cases / application services
  - Command handlers and query services
  - Transaction boundaries and orchestration
  - DTOs and mapping
- **Domain Layer**
  - Domain entities, aggregates, value objects
  - Domain rules and invariants
  - Domain events (optional)
- **Infrastructure Layer**
  - SQL Server data access (EF Core/repositories)
  - External system adapters (Integration Hub connectors)
  - Email provider integration (Exchange)
  - Hangfire job scheduling/execution
  - Observability plumbing implementations

Dependency direction rule:
Dependencies point **inward** (Infrastructure depends on Application/Domain; Domain has no dependency on Infrastructure).

C) DDD Boundary Application (Where Relevant)

Even within a single deployment unit, the solution applies domain boundaries to reduce coupling and keep business logic maintainable.

Indicative bounded contexts / domains:

- Inquiry Domain (submission, status lifecycle, SLA timestamps)
- Clearance Domain (orchestration, validations, decisions)
- Document Domain (generation, download rules, expiry)
- Agency/Tenant Domain (tenant identity, entitlements, batch behavior)
- Reporting Domain (query models, exports)
- Admin Configuration Domain (rules, fees, routing config)

Cross-domain interactions are performed through application services or domain events rather than direct data/table coupling.

D) CQRS-Light Pattern (Commands vs Queries)

To support near real-time dashboards and high performance without overcomplicating the architecture, the solution adopts **CQRS-light**:

- **Commands (Write path)**
  - Update operational state (Inquiry, Decision, etc.)
  - Enforce business rules
  - Trigger events for audit/notifications
- **Queries (Read path)**
  - Use dedicated query services optimized for filtering and aggregation
  - Read from reporting read models / SQL views where appropriate
  - Enforce tenant isolation server-side for portal views

CQRS-light does not imply microservices; it remains within the modular monolith with clear separation in code structure.

E) Cross-Cutting Implementation Enforcement

Cross-cutting concerns are enforced consistently across modules:

- **Identity & Authorization**
  - Policy enforcement at API boundary
  - Tenant claims enforced server-side in queries/commands
- **Audit Trail**
  - Audit.NET used for structured audit events on privileged operations
  - Criticality rules applied (critical events fail if audit persistence fails)
- **Logging & Tracing**
  - Serilog for structured logs, OpenTelemetry for traces/metrics
  - CorrelationId propagated end-to-end
- **Health Checks**
  - Standard liveness/readiness endpoints and dependency checks

F) Suggested Repository / Project Structure (Indicative)

A consistent project structure supports governance and maintainability:

- `/src/Presentation/*` (API host, portal integration endpoints)
- `/src/Modules/<ModuleName>/Application/*`
- `/src/Modules/<ModuleName>/Domain/*`
- `/src/Modules/<ModuleName>/Infrastructure/*`

- `/src/BuildingBlocks/*` (shared abstractions: correlation, auditing contracts, common primitives)
- `/src/Integrations/*` (external adapters grouped by entity/provider)
- `/tests/*` (unit, integration, functional)

Shared code is minimized; shared abstractions live in BuildingBlocks and are consumed through interfaces.

4.3.2 Technology stack

Solution approach is to leverage best combination of the enabling technology layers so that it drives for example:

- Solution deployment flexibility,
- Faster time to market when delivering solution features,
- Use of the latest technical advanced when it comes to application hosting approach,
- Implementation of continuous integration, continuous delivery,
- Broad community support behind the components chosen so that customer can get teams that continue to evolve or support the solution in future even without our team.

High level view of current set of technologies to be used in implementation of archiving solution are depicted in table below:

Layer	Component	Comment (Why Selected)
Presentation Layer	Angular (Latest LTS)	Selected for its maturity, long-term support, and wide enterprise adoption. Angular provides strong structure, built-in security features, and scalability suitable for complex role-based portals such as Web Portal and Admin Control.
Presentation Layer	HTML5 / CSS3 / TypeScript	Industry-standard web technologies that ensure compatibility, maintainability, and strong typing on the frontend, reducing runtime errors and improving developer productivity.
Application Layer	ASP.NET Core (.NET – Latest LTS)	Chosen as a stable, high-performance backend framework with native support for security, dependency injection, configuration management, and cross-platform hosting, making it suitable for enterprise and governmental systems.

Application Layer	C#	Selected for its strong typing, rich ecosystem, and deep integration with the .NET platform, enabling maintainable and robust implementation of business logic and background processing.
Application Layer	Hangfire	Used to support background and asynchronous processing without introducing additional infrastructure. Hangfire integrates natively with .NET and SQL Server, supports retries and monitoring, and fits well with a modular monolithic architecture.
Integration Layer	RESTful APIs over HTTPS	Selected as the most interoperable and widely supported integration mechanism for communication with external governmental and non-governmental entities, ensuring secure and standardized data exchange.
Integration Layer	.NET HTTP Client Libraries	Used for outbound integration calls due to native support, reliability, and ease of applying timeout, retry, and resiliency policies when communicating with external systems.
Data Layer	Microsoft SQL Server (Latest Supported Version)	Selected for its strong transactional consistency, enterprise-grade security features, mature tooling, and suitability for both operational workloads and reporting scenarios.
Data Layer	Entity Framework Core	Chosen to accelerate development through object-relational mapping, transaction management, and integration with .NET, while maintaining flexibility for optimized queries when needed.
Data Layer	Dapper (Optional)	Used selectively for performance-critical read scenarios where lightweight and fine-grained SQL control is required, particularly for reporting and dashboard queries.
Operational & Support Layer	Audit.NET	Selected as a specialized auditing framework for .NET that provides structured, consistent, and extensible audit logging aligned with compliance and governance requirements.

Operational & Support Layer	Serilog	Chosen for its structured logging capabilities, rich ecosystem of sinks, and strong support for contextual logging, enabling effective troubleshooting and diagnostics.
Operational & Support Layer	Seq	Selected as a centralized log storage and analysis platform that integrates seamlessly with Serilog and provides powerful querying and visualization for operational support teams.
Operational & Support Layer	OpenTelemetry	Adopted as a vendor-neutral standard for distributed tracing and metrics, enabling end-to-end observability and future flexibility in selecting telemetry backends.
Operational & Support Layer	ASP.NET Core Health Checks	Selected for its native integration with ASP.NET Core, allowing consistent implementation of liveness and readiness checks without additional tooling or infrastructure.
Operational & Support Layer	Microsoft Exchange (Gahez)	Used as the official outbound email channel to align with Gahez infrastructure standards and ensure controlled, trusted delivery of system notifications.
Security & Access	Identity Hub (OIDC / OAuth2 compatible)	Selected to centralize authentication and authorization, enforce consistent access policies, and support secure token-based access across portals and APIs.
DevOps & CI/CD	Azure DevOps	Chosen to provide integrated source control, CI/CD pipelines, and release management, supporting traceability, automation, and governance across environments.

For more information on OSS technologies used in the solution see Appendix - Use of Open Source.

4.3.3 Team Responsibilities

This section defines **responsibility boundaries** between **internal delivery teams**, **stakeholder teams**, and **external entities** involved in the implementation and operation of the solution.

The objective is to ensure **clear ownership, accountability, and coordination** across all parties.

A) Internal Delivery Teams (Implementation & Operations)

Team	Responsibility Scope
Full-Stack Development Team	End-to-end implementation of the solution including Web Portal, Admin Control, APIs, business logic, integrations, background processing, and data access. Owns feature development, code quality, and technical design within agreed architecture.
DevOps / Platform Team	CI/CD pipelines, environment provisioning, deployments, configuration management, monitoring setup, and operational readiness across all environments.
Security & Identity Team	Configuration and governance of Identity Hub, access policies, authentication/authorization rules, and security reviews of the solution.
Data & Reporting Team	Database schema design, reporting read models, performance tuning, data retention policies, and support for dashboards and exports.
Operations & Support Team	Runtime monitoring, incident management, health checks, log review, first-line operational support, and coordination of issue resolution.

B) Stakeholder Teams (Business & Governance)

Team	Responsibility Scope
Gahez Business Owners	Define business rules, policies, SLAs, and approval criteria. Validate functional behavior and accept delivered features.
Gahez Administrative Users	Operate Admin Control features, manage configurations, review dashboards, and oversee day-to-day system usage.
Gahez Compliance / Governance	Review audit trail, ensure regulatory compliance, and define data retention and access policies.

C) External Entity Teams (Integration Participants)

Entity	Responsibility Scope
Governmental Systems Teams	Provide integration endpoints, data contracts, and test environments; respond to inquiries as per agreed protocols.

Non-Governmental Service Providers	Support external validations or data enrichment through exposed APIs and agreed SLAs.
ESC Payment Gateway Team	Provide payment APIs, integration documentation, and operational support for payment-related workflows.
Email Infrastructure Team (Exchange)	Ensure availability and configuration of outbound email services used by the solution.

Responsibility Demarcation Principles

- **Internal teams own implementation and operation** of the solution.
- **Stakeholder teams define policies and validate outcomes**, not technical design.
- **External entities own their systems and interfaces**, not internal workflows.
- All integrations are governed by clearly defined contracts and SLAs.

5. Deployment View

5.1 Environments

Due to the size and criticality of the solution, and its continuous evolution during production operation, multiple deployment environments are required to ensure proper segregation of activities, controlled change management, and operational stability.

The use of multiple environments enables:

- Parallel development and testing activities
- Progressive validation of functional, integration, performance, and availability requirements
- Risk reduction before introducing changes into production

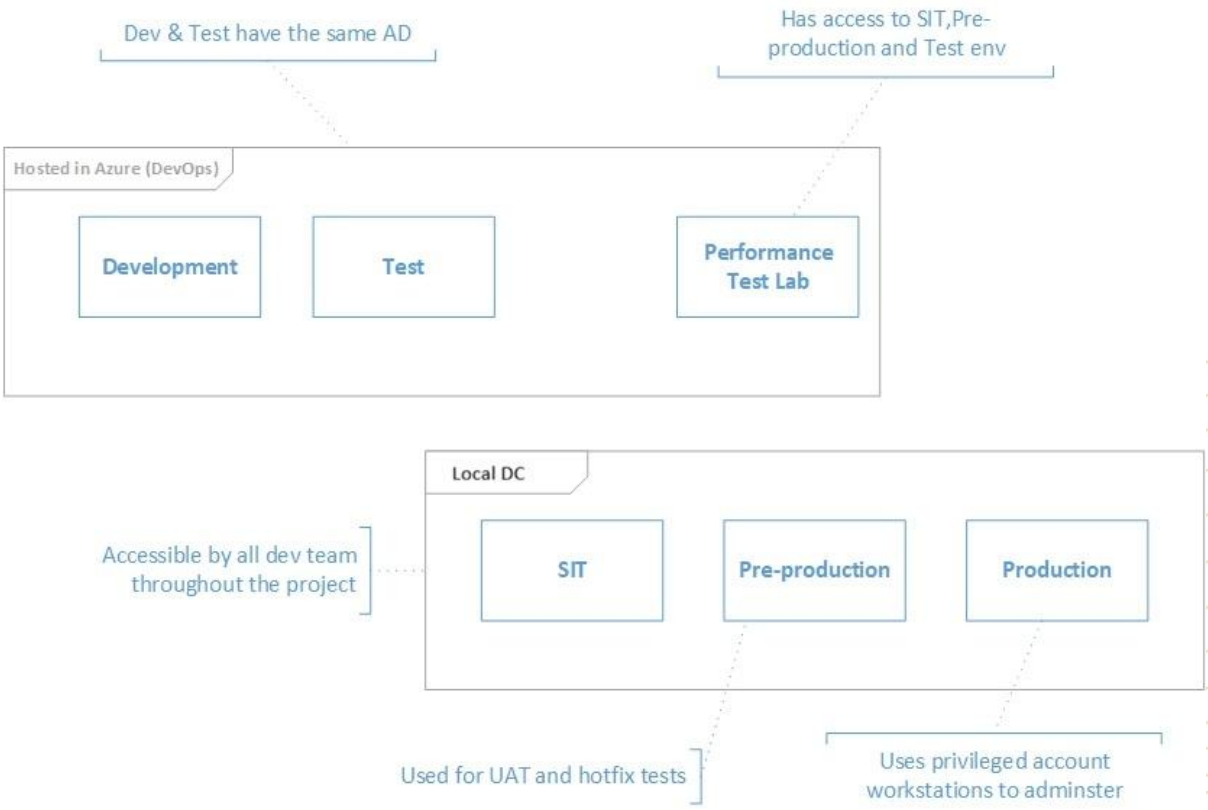

Figure 19 Solution environments and development tools deployment

The solution relies on a **multi-environment deployment model**, where each environment serves a specific purpose within the delivery and operational lifecycle. All environments are managed through centralized configuration management and

automated deployment mechanisms to ensure consistency and traceability across releases.

The solution is deployed across **five logical environments**, as described below.

Environment	Hosting Approach	Sizing Approach
Development Environment	Hosted in a non-production environment intended for active development and functional validation. High availability configuration is not required in this environment, as it is not used to validate availability scenarios.	Minimal infrastructure sufficient to support development activities and basic functional testing.
Testing Environment	Hosted in a non-production environment used to validate solution functionality and availability configurations. High availability settings are enabled to verify behavior under redundancy scenarios.	Minimal infrastructure capable of supporting high availability configuration, without full production scale.
System Integration Testing (SIT)	Hosted in a controlled environment with access to test instances of integrated external systems. High availability configuration is enabled to support integration validation.	Reduced-scale replica of the production environment, closely matching production configuration.
User Acceptance Testing (UAT) / Pre-Production	Hosted in a production-like environment with high availability configuration. Used to validate configuration changes, hotfixes, and business acceptance prior to production deployment.	Recommended to closely match production environment in both configuration and sizing.
Production Environment	Hosted in a production-grade setup with full scale and high availability configuration. Designed to support live workloads, operational monitoring, and business continuity requirements.	Full production scale, sized to handle peak operational workloads and growth expectations.

5.2 Host configuration

The hosting configuration of the solution is standardized to ensure consistency, security, and operational stability across all deployment environments. All solution

components are hosted on **Windows-based servers**, aligned with enterprise infrastructure standards and operational requirements.

Operating System Standard

Operating System	Version	Description
Windows Server	Latest Long-Term Servicing Channel (LTSC) version supported by the infrastructure	Used to host all solution components, including application services, operational databases, reporting components, and supporting infrastructure services.

The selected operating system provides a secure, stable, and enterprise-grade platform for hosting the solution while ensuring long-term support, security updates, and compatibility with modern tooling.

Post-Installation Host Configuration Requirements

After operating system installation, the following configuration steps are applied consistently across all hosts:

- **Time Synchronization**
  All hosts are configured to synchronize system clocks using a centralized and reliable time synchronization service to ensure accurate logging, auditing, and event correlation across all solution components.
- **Static Network Configuration**
  Static IP addresses are assigned to all hosts to support stable service identification, secure communication, and certificate-based trust between internal solution components.
- **Hostname Standardization**
  Hostnames are assigned using a standardized naming convention and are used consistently for monitoring, logging, security policies, and operational management.
- **Network Security Configuration**
  Host-level firewalls are enabled and configured to allow only explicitly required inbound and outbound traffic based on approved deployment and security guidelines.
- **Software Installation Controls**
  All software installations follow clean-source principles, ensuring that installation packages and binaries are obtained only from trusted, verified, and approved sources.

- **Operating System Hardening**
  The Windows operating system is hardened in accordance with the latest Microsoft security baselines and industry best practices, including:
  - Disabling unnecessary services and features
  - Applying secure configuration policies
  - Enforcing least-privilege access controls
- **Firewall Enforcement**
  Windows Firewall remains enabled on all hosts, with explicit and reviewed rules configured to support solution components and integrations securely.

Hosting Configuration Objectives

This hosting configuration approach ensures that:

- The solution operates on a secure, supported, and maintainable runtime platform
- Security risks related to OS misconfiguration are minimized
- Operational troubleshooting and maintenance are simplified
- Hosting environments remain consistent across all lifecycle stages

6. Operation View

6.1 DevOps Enablement and Operational Tooling

From an operational tooling perspective, the solution adopts an integrated DevOps enablement model that supports the continuous delivery and stable operation of the system across its lifecycle.

The operational "outer loop" relies on the following capabilities:

- **Continuous Integration and Continuous Delivery (CI/CD)**
  Automated pipelines are used to build, validate, package, and deploy solution components in a repeatable and controlled manner across environments.
- **Structured Source Control and Branching Strategy**
  A disciplined branching and merging strategy is applied to support parallel development, defect resolution, and controlled releases while preserving code stability and traceability.
- **Automated Quality Gates**
  Quality validation is enforced as part of the pipeline execution to prevent defective or non-compliant changes from progressing through environments.
- **Test Automation Across Multiple Dimensions**
  Automated testing is applied at different levels to ensure solution correctness and stability, including:
  - Unit testing
  - Functional testing
  - Integration testing
  - Performance testing
  - Accessibility testing
  - Compatibility testing
- **Build and Release Validation Controls**
  Each build and release undergoes automated validation checks prior to promotion, ensuring that only verified artifacts are eligible for deployment.
- **Open-Source Compliance Verification**
  Where open-source components are used, automated checks are applied during the build and release process to ensure license compliance and reduce legal and operational risk.
- **Operational Monitoring and Health Checks**
  Automated and manual monitoring mechanisms are used to detect service health issues, failures, or anomalies that may affect system stability or user experience.
- **Event Tracing and Telemetry Collection**
  Events occurring within the system are traced and collected to support both technical diagnostics and business-level analysis, enabling proactive issue identification and continuous improvement.

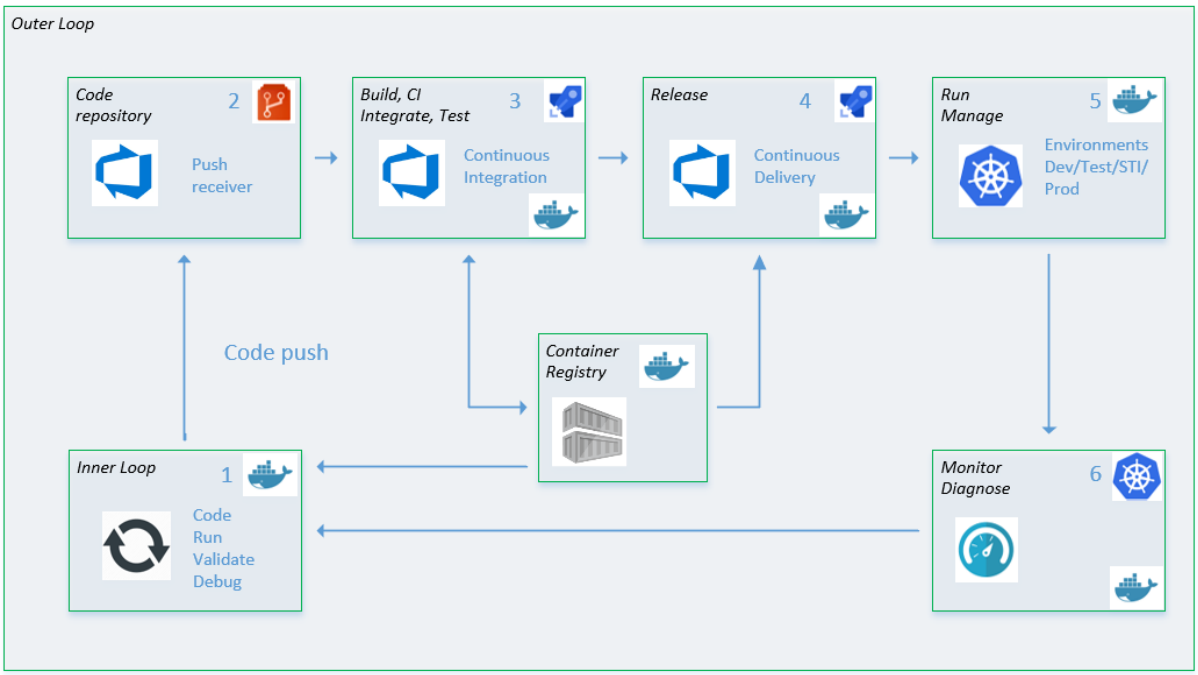

Figure 20DevOps Outer Loop

6.2 Inner Development Loop and Engineering Practices

The "inner loop" of the DevOps process focuses on developer productivity, code quality, and early defect detection before changes reach shared environments.

The following practices are applied during solution development:

- **Static Code Analysis**
  Static analysis tools are integrated into the development workflow to detect code quality issues, security risks, and maintainability concerns early in the development cycle.
- **Peer Code Review**
  Code changes are subject to peer review prior to integration, ensuring adherence to architectural guidelines, coding standards, and functional correctness.
- **Coding Standards and Best Practices**
  The solution is implemented following established coding standards and best practices to support maintainability and efficient troubleshooting in production environments.

  These practices include:

- Clear and meaningful code comments when additional context is required
- Consistent naming conventions across all code elements
- Use of a common language for code, comments, and identifiers to ensure shared understanding across teams

- **Troubleshooting-Oriented Design**
  Code structure and logging practices are designed to simplify debugging and root-cause analysis during runtime, reducing mean time to resolution (MTTR) when issues occur.

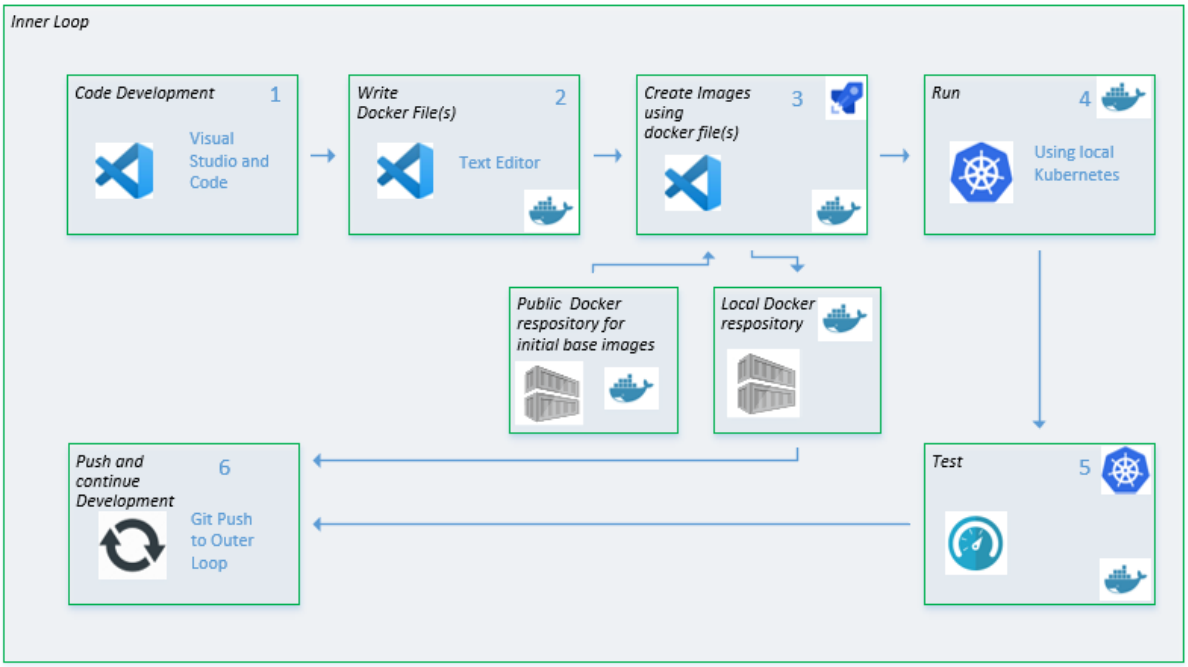

Figure 21 DevOps strategy Inner Loop

6.3 Feedback Loop Between Operations and Development

Operational insights gathered during system runtime—such as performance metrics, error trends, and incident reports—are fed back into the development lifecycle.

This feedback loop ensures that:

- Production issues result in corrective actions or enhancements
- Performance bottlenecks are addressed through architectural or configuration improvements
- Operational risks are mitigated proactively rather than reactively

This continuous feedback mechanism enables the solution to evolve while maintaining operational stability and predictable behavior.

7. Hardware Sizing

7.1 Assumptions

This section defines the **quantitative assumptions** used as the basis for estimating the initial hardware sizing of the solution for the **first year of operation**. All values are indicative and will be validated and refined based on real usage metrics after go-live.

User Volume Assumptions (Year 1)

User Type	Assumption
Travelers	**50,000** unique users per year
Agencies	**100** registered agencies
Administrative Users	**10** active admin users

Concurrency Assumptions

Item	Assumption
Peak concurrent traveler sessions	**300** concurrent users
Peak concurrent agency users	**30** concurrent users
Peak concurrent admin users	**5** concurrent users
Total peak concurrent users	**~335** concurrent users

Assumes ~0.7% concurrency ratio for travelers during peak periods.

Transaction & Request Volume Assumptions

Metric	Assumption
Average inquiries per traveler	**1-2** per year
Total inquiries per year	**~100,000** inquiries
Average daily inquiries	**~205** inquiries/day
Peak daily inquiries (seasonal)	**~1,000** inquiries/day
Average API requests per inquiry	**8-12** requests
Peak requests per second (RPS)	**15-20 RPS** (internal, excluding external integrations)

Agency Batch Processing Assumptions

Metric	Assumption
Agencies using batch submissions	**~60%** of agencies
Average batch size	**50-200** inquiries
Maximum batch size (soft limit)	**500** inquiries
Max concurrent batch jobs	**5-8** concurrent jobs
Background workers allocated to batch jobs	**≤30%** of available workers

Background Processing Assumptions

Metric	Assumption
Background job types	Processing, notifications, reporting
Average background jobs per inquiry	2-3 jobs
Total background jobs per day	~500-700 jobs/day
Job retry attempts	up to 3 retries
Job execution timeout	≤2 minutes per job

Integration Assumptions

Metric	Assumption
External integrations per inquiry	1-3 systems
Average external API latency	1-5 seconds
External API timeout	10-15 seconds
Integration retry attempts	up to 2 retries
Percentage of async integration flows	≥70%

Data Volume Assumptions (Year 1)

Data Type	Assumption
Operational records	~75,000-100,000 records
Audit trail records	~10-15× operational volume (≈ 1-1.5M records)
Average audit record size	1-2 KB
Estimated audit data size (Year 1)	~2-3 GB
Stored documents	Not stored persistently (generated on demand)

Reporting & Dashboard Assumptions

Metric	Assumption
Dashboard refresh delay	1-5 minutes
Concurrent dashboard viewers	10-20 users
Export operations per day	≤20 exports/day
Export size	≤50,000 rows per export

Availability & SLA Assumptions

Metric	Assumption
Target availability (production)	99.5%
Planned maintenance window	Monthly, off-peak
Target API response time (internal)	< 1 second (p95)
SLA excludes	External integration latency

7.2 Production environment

Below table represents requirements for production environment (mention DC approach used active-active or active-passive).

Role	Servers	CPU	Memory	Storage	Network	Software
Presentation Layer (Web Hosting)	**2 x VMs**	**4 vCPUs**	**16 GB**	**150 GB (OS) + 50 GB (Web static files) SSD**	**1 Gbps**	Windows Server (latest), IIS (static hosting / reverse proxy as applicable)
API / Application Layer	**2 x VMs**	**4 vCPUs**	**16 GB**	**150 GB (OS) + 100 GB (App/Temp) SSD**	**1 Gbps**	Windows Server (latest), .NET (latest LTS), IIS/Hosting bundle
Background Processing (Hangfire Workers)	**1 x VM**	**4 vCPUs**	**16 GB**	**150 GB (OS) + 100 GB (Jobs/Temp) SSD**	**1 Gbps**	Windows Server (latest), .NET (latest LTS), Hangfire Server
Database Layer (SQL Server – HA within single DC)	**2 x VMs**	**6 vCPUs**	**32 GB**	**150 GB (OS) + 250 GB (Data files) SSD + 100 GB (Log files) SSD + 500 GB (Backups/Archive)**	**1–10 Gbps**	Windows Server (latest), Microsoft SQL Server (latest supported), HA configuration (AlwaysOn/cluster per infrastructure standards)
Centralized Logs (Seq)	**1 x VM**	**2 vCPUs**	**8 GB**	**150 GB (OS) + 300 GB (Log store) SSD**	**1 Gbps**	Windows Server (latest), Seq
Telemetry / Monitoring Services *(optional but recommended)*	**1 x VM**	**2 vCPUs**	**8 GB**	**150 GB (OS) + 100 GB (Telemetry buffer) SSD**	**1 Gbps**	Windows Server (latest), OpenTelemetry Collector / monitoring agents

7.3 Pre-Production Environment (Optional / Preferred)

The **Pre-Production environment** is an **optional but strongly recommended** environment that mirrors the Production setup in terms of **configuration, topology, and capacity**. Its primary purpose is to validate system behavior under production-like conditions before any release is promoted to the live environment.

7.4 Staging / System Integration Testing (SIT) Environment

The **Staging (SIT) environment** is used to validate **end-to-end system integration** across internal modules and external entities before promoting changes to Pre-Production or Production environments.

Role	Servers	CPU	Memory	Storage	Network	Software
Presentation Layer (Web Hosting)	**1 x VM**	**2 vCPUs**	**8 GB**	**150 GB (OS) + 30 GB (Web static files/cache) SSD**	**1 Gbps**	Windows Server (latest), IIS
API / Application Layer	**1 x VM**	**4 vCPUs**	**8 GB**	**150 GB (OS) + 80 GB (App/Temp) SSD**	**1 Gbps**	Windows Server (latest), .NET (latest LTS), IIS/Hosting bundle
Background Processing (Hangfire Workers)	**1 x VM**	**2 vCPUs**	**8 GB**	**150 GB (OS) + 50 GB (Jobs/Temp) SSD**	**1 Gbps**	Windows Server (latest), .NET (latest LTS), Hangfire Server
Database Layer (SQL Server)	**1 x VM**	**4 vCPUs**	**16 GB**	**150 GB (OS) + 200 GB (Data files) SSD + 100 GB (Log files) SSD**	**1 Gbps**	Windows Server (latest), Microsoft SQL Server (latest supported)
Centralized Logs (Seq) *(Optional)*	**1 x VM**	**2 vCPUs**	**4 GB**	**150 GB (OS) + 100 GB (Log store) SSD**	**1 Gbps**	Windows Server (latest), Seq
Telemetry / Monitoring Services *(Optional)*	**Shared / Optional**	—	—	—	—	OpenTelemetry agents (can share with App VM)

The SIT environment is intentionally sized smaller than Production and Pre-Production, while preserving architectural consistency to ensure reliable integration validation.

8. Appendices

8.1 Glossary of Key Terms

Below table details the key terms, acronyms and abbreviations used in this document:

Term	Description
Security Clearance Gateway	The solution described in this document, responsible for managing security clearance inquiries, processing, integrations, and approvals.
Web Portal	User-facing web application used by travelers and agencies to submit inquiries, track status, and download approved documents.
Admin Control Portal	Administrative web application used by Gahez staff and decision makers to configure rules, manage operations, and review dashboards.
Traveler	An individual end user submitting a security clearance inquiry through the Web Portal.
Agency	A registered organization authorized to submit individual or batch clearance inquiries on behalf of travelers.
Administrative User	A privileged user responsible for system configuration, monitoring, and operational control.
Decision Maker	A senior or authorized user who accesses dashboards and reports for oversight and decision support.
Inquiry	A request submitted to the system to obtain security clearance approval for a traveler.
Batch Submission	A group of inquiries submitted together by an agency in a single operation.
Clearance Processing	The business workflow responsible for validating inquiries, coordinating integrations, and determining final outcomes.
Document Issuance	The process of generating an official clearance document dynamically upon successful approval.
QR Verification	A feature allowing authorized users to validate clearance-related data using a secure QR code.
Integration Hub	The logical module responsible for managing communication with external governmental and non-governmental systems.
External Governmental Systems	State or governmental systems integrated with the solution to validate or respond to clearance inquiries.
External Non-Governmental Service Providers	Third-party systems providing supplementary validation or services related to clearance processing.

ESC Payment Gateway	External payment system used to process fees associated with clearance inquiries.
API (Application Programming Interface)	A set of endpoints exposed by the solution to enable system-to-system communication.
Partner/System APIs	APIs exposed specifically for external agencies or systems to integrate with the solution.
API Entry Layer	The secured entry point through which external API consumers access the solution, implemented via reverse proxy, WAF, or API management tools.
Presentation Layer	Architectural layer responsible for user interaction, including Web Portal and Admin Control.
Application / Processing Layer	Architectural layer responsible for business logic, workflows, and background processing.
Integration Layer	Architectural layer handling communication with external systems and services.
Data Layer	Architectural layer responsible for data persistence, audit storage, and reporting data.
Operational Monitoring & Supportive Layer	Layer providing logging, tracing, audit trail, notifications, and health checks.
Hangfire	Background job processing framework used to execute asynchronous and scheduled tasks.
Audit Trail	A secure, immutable record of significant business and administrative actions within the solution.
Logging	Recording of technical events and application behavior for troubleshooting and analysis.
Tracing	Tracking of request flows across components for performance and diagnostics.
Health Check	Mechanism used to monitor the availability and status of system components.
CI/CD	Continuous Integration and Continuous Deployment practices used to build, test, and release the solution.
SIT (System Integration Testing)	Environment used to validate integrations between internal modules and external systems.
Pre-Production Environment	Production-equivalent environment used for final validation before live deployment.
Production Environment	Live environment where real users and operational data are handled.
HA (High Availability)	Architectural approach to ensure system availability through redundancy.
SLA (Service Level Agreement)	Defined performance and availability targets for system operations.

8.2 Use of Open Source

This section lists all **open-source software components** used by the solution. These components are selected based on maturity, community support, security posture, and compatibility with the selected technology stack.

Layer	Open Source Component	Purpose	License (Indicative)
Presentation Layer	Angular	Frontend framework for Web Portal and Admin Control Portal	MIT
Presentation Layer	TypeScript	Strongly typed language for frontend development	Apache 2.0
Application Layer	ASP.NET Core	Backend framework for APIs and application services	MIT
Application Layer	Entity Framework Core	ORM for database access and transaction management	MIT
Application Layer	Hangfire (Community Edition)	Background job processing and scheduling	LGPL
Data Layer	Dapper	Lightweight data access for optimized read queries	Apache 2.0
Operational & Support Layer	Audit.NET	Structured audit trail framework	MIT
Operational & Support Layer	Serilog	Structured logging framework	Apache 2.0
Operational & Support Layer	OpenTelemetry SDK	Distributed tracing and metrics instrumentation	Apache 2.0
Operational & Support Layer	ASP.NET Core Health Checks	Health monitoring framework	MIT

Notes:

- All open-source components are widely adopted and actively maintained.
- Licenses are permissive and compatible with commercial and governmental usage.
- License versions will be validated during implementation and release phases.

8.3 Licenses and subscriptions

For on-premise licenses, below table contains required licenses per layer:

Product	License Type	Notes (Aligned with HW Specs)
Windows Server	**Standard Edition**	Suitable for all application, presentation, background processing, logging, and monitoring VMs. The environment uses a limited number of VMs per host with no heavy virtualization density; Datacenter edition is not required.
Microsoft SQL Server	**Standard Edition (Per Core)**	Sufficient for the expected Year-1 workload, data volume, and performance profile. Supports SQL HA (Always On availability groups with basic features) within a single data center. Both SQL nodes must be licensed.

8.4 Technologies & Techniques Used

8.4.1 Angular

Description
Angular is a modern, component-based front-end framework used for building rich, responsive, and secure web applications.

Why Angular is Suitable

- Provides a **structured and opinionated framework**, which promotes consistency across large teams.
- Strong support for **enterprise-scale applications** with clear separation of concerns.
- Built-in features for **routing, form validation, dependency injection, and internationalization**.
- Strong alignment with **security best practices**, including protection against XSS and CSRF.
- Long-term support (LTS) releases ensure **stability and predictable upgrades**.

Typical Use Cases

- User-facing web portals
- Administrative dashboards
- Role-based and multi-tenant web applications

Reference
https://angular.io/docs

8.4.2 .NET Core (ASP.NET Core)

Description
.NET Core (ASP.NET Core) is a cross-platform, high-performance framework for building modern backend applications, APIs, and services.

Why .NET Core is Suitable

- Designed for **high performance and scalability**, suitable for systems with variable load.
- Native support for **RESTful APIs**, background processing, and secure authentication mechanisms.
- Strong tooling and ecosystem for **enterprise development**, testing, and observability.
- Supports **modular and layered architectures** such as Clean Architecture.
- First-class integration with modern DevOps pipelines and cloud-native environments.

Typical Use Cases

- Business logic and application services
- API development
- Background and asynchronous processing
- Integration services

Reference
https://learn.microsoft.com/dotnet/core/

8.4.3 Microsoft SQL Server

Description
Microsoft SQL Server is a relational database management system designed for reliability, security, and performance in enterprise environments.

Why SQL Server is Suitable

- Provides **strong transactional consistency** and reliable data integrity.
- Advanced security features including **encryption, auditing, and access control**.
- Mature tooling for **backup, recovery, monitoring, and performance tuning**.
- Supports **high availability and disaster recovery** configurations.
- Well-suited for systems requiring structured data, reporting, and auditability.

Typical Use Cases

- Operational data storage
- Authoritative persistence

- Reporting and read-optimized queries

Reference

https://learn.microsoft.com/sql/sql-server/